

Using Generative AI to Explore the Limits of Jet Tagging

by
Nishank Gite



5/12/2024

Doctor Benjamin Nachman, Research Advisor

Professor Uros Seljak, Faculty Advisor



A thesis submitted in partial fulfillment
of the requirements for the
Degree of Bachelor of Arts with Honors
in Physics

University of California, Berkeley

Berkeley, California

May 13, 2024

Acknowledgements

First and foremost I am extremely grateful to Dr. Benjamin Nachman for his priceless guidance and mentorship during my research journey and beyond. Dr. Nachman not only welcomed me into his research group for the Fall Semester, but also agreed to be my advisor for my senior honors thesis on such short notice for which I am beyond thankful. His ability to simplify complex concepts and his prompt responses to my questions on Slack have been exceedingly beneficial. I am deeply indebted to him for his commitment to our weekly meetings and his efforts to seamlessly integrate me into his research group. His dedication and support have played a pivotal role in the achievements of my academic and research pursuits and it has been an honor to learn from him for the past year.

Furthermore, I am exceptionally thankful to Dr. Vinicius Mikuni for his invaluable support during my research. Dr. Mikuni's persistent readiness to set up meetings and willingness to assist me through Slack ensured that I had a clear path and guidance at every step of my work. His commitment demanded a substantial amount of his time, and for this, I am truly grateful. His valuable insights and expertise played a crucial role in the accomplishment of this thesis, and I am sincerely appreciative of all his assistance.

Finally, I want to thank Professor Uros Seljak for agreeing to be my official faculty advisor for my thesis despite asking extremely last second, and without whom this would not have happened.

Abstract

This study delves into the utilization of Generative AI in determining the theoretical upper limits of classifier performance in jet tagging. By utilizing the Neyman-Pearson Lemma, which is the uniformly most powerful method for hypothesis testing, a robust framework is established for comparing classifier performances. The mathematical underpinning, based on the likelihood ratio defined by $\mathcal{L}(x) = \frac{p(x|\theta_0)}{p(x|\theta_1)}$, is extensively examined in order to highlight the theoretical capabilities of our classifiers but also assists in optimizing them to minimize false positives and maximize true positives effectively. Given that the probability distribution for real data is unknown, our study utilizes diffusion generative models trained on collider jet data to produce identical synthetic jet data. Generated and Real jet data similarity is confirmed through residual feature comparisons, and using the generated probability distribution we calculate log-likelihoods close to zero for the gluon and top models on their respective data sets (indicating high classification accuracy), while significant negative values for cross-category applications validate the discriminative capabilities of the models. This alignment is further supported by ROC curves for the likelihood ratio and classifiers, showcasing substantial improvements in classifier performance with the incremental inclusion of transformers. These enhancements are apparent from comparative AUC (Area Under Curve) metrics, confirming the efficacy of our methodology. Additionally, ongoing enhancements based on iterative feedback from ROC analysis suggest a promising trajectory towards achieving nearly optimal classifier performance in the future.

Contents

Acknowledgements	i
Abstract	ii
1 High Energy Physics	1
1.1 Thesis Structure	1
1.2 Introduction	2
1.3 Motivations and Importance	3
1.4 High Energy and Collider Physics	4
1.4.1 High Energy Theory	5
1.4.2 Colliders	11
1.5 Research Objectives	13
2 Machine Learning	15
2.1 Introduction to Artificial Intelligence and Machine Learning	15
2.1.1 Modeling	18
2.1.2 Classifiers	21
2.1.3 Generators	24
2.2 Deep Learning	28
2.3 Tools and Frameworks	31

2.4	Research Applications	32
3	Jet Tagging	33
3.1	Design and Implementation	33
3.2	Results	36
3.3	Discussion	40
3.3.1	Diffusion Model Certainty	40
3.3.2	Likelihoods	41
3.3.3	Classifiers	43
3.4	Future Research	45
	Conclusion	47
A	Equations Appendix	49
A.1	Likelihood Ratio	49
B	Code Appendix	51
B.1	Classifier Model Code	51
B.2	Diffusion Generative Model Code	52

List of Figures

1.1	The Standard Model of particle physics. [1]	5
1.2	Illustration of various First-Order Feynman diagram vertices.	7
1.3	Illustration of an example hadronization process in high energy physics.	9
2.1	Venn Diagram explaining the differences between AI, ML and Deep Learning. [2]	15
2.2	Illustration of how Generators and Classifiers are different in their learning methodology. [3]	17
2.3	Within machine learning, this image depicts how gradient descent works. [4]	19
2.4	Normalization of data. [5]	20
2.5	The primary methods to approach classification in machine learning. [6]	21
2.6	Overfitting and underfitting in the context of classification. [7]	23
2.7	An overview of all the primary kinds of generative models used today. [8]	24
2.8	Specific depiction of the Diffusion Model used in this research. [9]	27
2.9	Specific depiction of the Diffusion Model used in this research. [10]	30
3.1	ROC curves for numerous high-performing jet tagging models. [11]	34
3.2	Plots of Generated vs. Real Particle and Jet Features for both Tops and Gluons, created by Dr. Vinicius Mikuni.	36

3.3	Plots of reconstructed Gaussian's for comparisons of the forward and backwards transformation of the diffusion model, created by Dr. Vinicius Mikuni.	37
3.4	These are heatmaps depicting the two model probabilities with the two different datasets across the particles and jets.	38
3.5	This image depicts the classifier model architecture, the code for which can be found in the Code Appendix B.1.	38
3.6	ROC curves for generated data with varying numbers of transformers.	39
3.7	ROC curves for real data with varying numbers of transformers.	39

Chapter 1

High Energy Physics

1.1 Thesis Structure

The initial chapter will offer a thorough overview of high energy physics, with a specific focus on the crucial role of collider physics in enhancing our comprehension of the fundamental building blocks of matter. Various types of particle accelerators, such as linear accelerators, cyclotrons, and synchrotrons, will be examined in terms of their operational principles and their utilization in accelerating particles to high energies for collision experiments. Following this, the discussion will shift towards particle interactions, explaining the intricacies of Feynman diagrams as a tool for visualizing and calculating the probabilities of particle processes. The chapter will conclude with an exploration of hadronization - the process through which quarks and gluons transition into jets of hadrons, a phenomenon that can be observed in particle collider data.

Moving forward from the foundational physics covered in Chapter 1, the subsequent chapter will delve into the realm of machine learning (ML). An overview of ML will be presented, encompassing its fundamental concepts and methodologies, as well as the evolution towards deep learning algorithms that have transformed data-driven research. The focus will

be specifically on the application of these techniques within the realm of high energy physics, analyzing how they can be utilized to study and interpret jets resulting from hadronization. The objective is to clarify the methodological foundations that will prepare readers for the intricate analysis presented in the following chapter.

With the necessary groundwork laid out, Chapter 3 will present the original research findings. The implementation of the aforementioned ML and deep learning techniques on jet data will be discussed, providing a detailed overview of the computational experiments carried out, the results achieved, and their significance within the broader scope of particle physics research. The chapter will wrap up with a discussion on the implications of these findings, both for the field of high energy physics and the evolving convergence with advanced computational methodologies. Potential future directions for extending this research will also be contemplated.

1.2 Introduction

The field of elementary particle physics is often traced back to the identification of the electron a century ago. Over the subsequent five decades, a series of new particles were uncovered, primarily through investigations involving cosmic rays, which were the primary source of extremely high-energy particles at that time. These discoveries would lead the discipline to experience significant growth post-1950, spurring the advancement of high-energy accelerators, which produced concentrated and regulated beams of particles with known energy levels. During the early decades of the twentieth century, the energy levels achieved by particle accelerators were limited to a mere few million electron volts. [12] Consequently, the resolution of these accelerators was so inadequate that protons and neutrons were considered to be elementary particles with no internal structure. However, in the present era, advancements in accelerator technology have significantly improved the resolution, enabling

scientists to identify quarks and leptons as the fundamental constituents of matter.

1.3 Motivations and Importance

As accelerator technology continued to improve numerous experiments came to fruition, and eventually the ATLAS collaboration was formed in the late 1990s. Furthermore, under the European Council for Nuclear Research (CERN), the Large Hadron Collider (LHC) was finally constructed and is the world's largest and most powerful particle accelerator. LHC gave birth to a new age of High Energy Physics, providing capabilities at the TeV energy scale, and it was during Run 1 of the LHC that people made one of the most groundbreaking discoveries of the 21st century: the discovery of the Higgs Boson.

With the discovery of the Higgs boson in 2012, the ATLAS experiment accomplished one of the most significant breakthroughs in the history of experimental particle physics. Twelve years after its discovery there are still countless questions, and learning more about Higgs interactions can reveal some of the Standard Model's greatest mysteries. As LHC runs continue and datasets evolved to grow even larger, it has become harder to distinguish subtle patterns in data that may infer rare undiscovered processes, and thus with the advent of machine learning and its proven capability to determine complex patterns in data, it has become popular contention that the integration of machine learning techniques in particle physics opens up potential avenues to explore phenomena beyond our current theoretical framework, especially with further analysis of the Higgs Boson. [13] Improving our ability to use machine learning for future data runs is relevant because proper identification of particles is especially crucial for enhancing the efficiency, sensitivity and purity of data samples used in Higgs Boson property measurements. Optimizing models has become a top priority for many high energy physicists, because Run 4 + 5 data is predicted to be much larger than previous Run data, and machine learning models are the future, as they are significantly more

effective than cut-based methods for differentiating between signal and background. Just as Run 1, 2 and 3 data spearheaded the discovery of the Higgs Boson alongside numerous other particle interaction processes, principles and statistics, there are still numerous mysteries in Particle Physics, and Run 4 and 5 data holds the potential for answering fundamental questions about matter-antimatter asymmetry, and even Dark Matter candidates.

The main objectives of High Energy physics involve investigating fundamental inquiries regarding the Universe, yet the additional advantages of such studies have resulted in advancements in technologies that have a substantial influence on daily life. High-energy physics research has played a direct role in enhancing cancer treatment, such as through proton therapy that reduces harm to nearby healthy tissues in contrast to conventional X-ray radiation therapy, as well as PET scans (Positron Emission Tomography) that originated from particle detection technologies and are essential for disease diagnosis and monitoring. [14] The future for high energy physics is promising, with many breakthroughs on the horizon.

1.4 High Energy and Collider Physics

High-energy physics is a field of study within physics that focuses on investigating the fundamental building blocks of matter and the interactions between them. Its primary objective is to comprehend the fundamental principles that govern the universe on a microscopic level. The ultimate aim is to develop a comprehensive and cohesive explanation for the four fundamental forces that are observed in nature: gravity, electromagnetism, strong nuclear force, and weak nuclear force.

1.4.1 High Energy Theory

The Standard Model

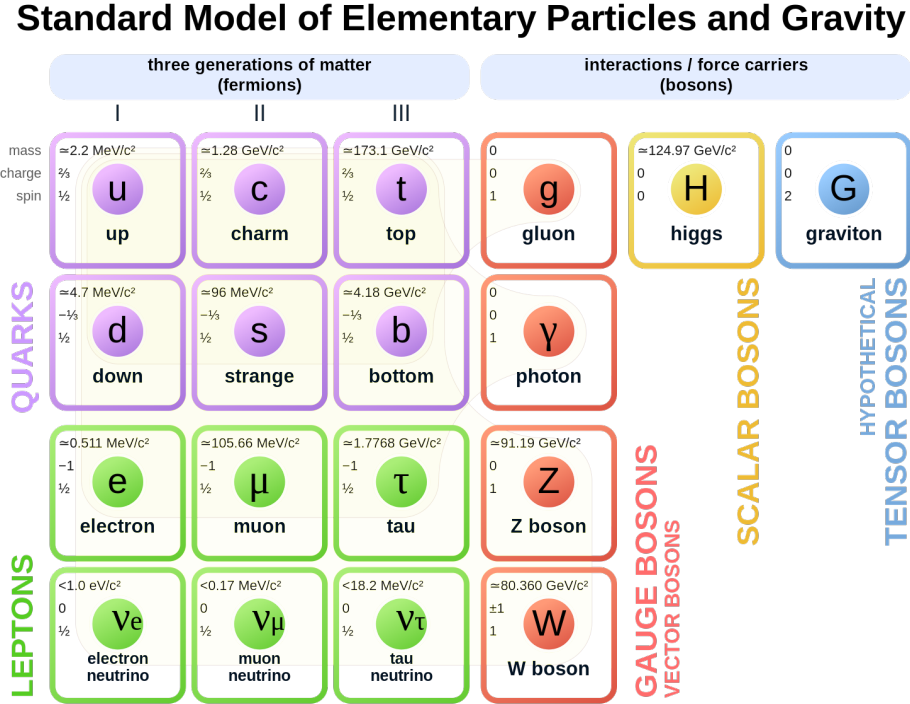


Figure 1.1: The Standard Model of particle physics. [1]

The Standard Model of particle physics is a widely accepted theory that explains the fundamental particles and forces in the universe, with the exception of gravity. It classifies all known subatomic particles into two main groups: fermions and bosons.

Fermions are considered the building blocks of matter and adhere to the Pauli exclusion principle, which states that two identical fermions cannot occupy the same quantum state simultaneously. Fermions are further divided into two categories.

The first category is quarks, which come in six different types or flavors. These quarks are organized into three generations. Each quark carries a fraction of the electric charge, either $+\frac{2}{3}$ (the upper three purple quarks) or $-\frac{1}{3}$ (the lower three purple quarks). The first

generation includes the 'up' and 'down' quarks, which are the most stable and form the protons and neutrons. The second generation consists of the 'charm' and 'strange' quarks, which are heavier and less stable. [1] The third generation comprises the 'top' and 'bottom' quarks, which are the heaviest and least stable.

The second category of fermions is leptons, which also come in three generations. Leptons include both charged leptons (electron, muon, and tau) and neutral neutrinos (electron neutrino, muon neutrino, and tau neutrino). Charged leptons have a charge of -1 , while neutrinos are electrically neutral.

Bosons, on the other hand, are particles that act as carriers of the fundamental forces in the universe, excluding gravity. They play a crucial role in mediating these forces. The bosons included in the Standard Model are:

1. Gluon (g): It carries the strong force, which is responsible for holding quarks together within protons, neutrons, and other hadrons.
2. Photon (γ): It carries the electromagnetic force, which is responsible for electromagnetic waves and interactions between charged particles.
3. W and Z Bosons: These bosons mediate the weak nuclear force, which is involved in processes like radioactive decay and neutrino interactions.
4. Higgs Boson (H): It is associated with the Higgs field, which imparts mass to particles that interact with it.

We should also note that the Standard Model does not include the graviton (G), which is a hypothetical particle that is believed to mediate the force of gravity.

Feynman

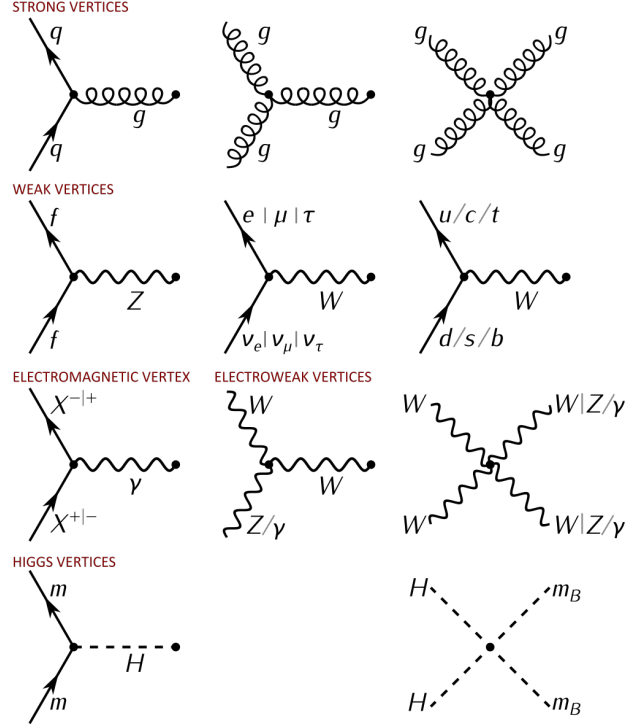


Figure 1.2: Illustration of various First-Order Feynman diagram vertices.
[15]

The most fundamental aspect of high energy physics boils down to Feynman diagrams and Figure 1.2 illustrates various first-order Feynman diagrams, encompassing Quantum Chromodynamics, which is the theory describing the strong interactions of quarks and gluons. Within Feynman diagrams, there exists three types of lines, each of which can be described as follows:

- **Gluon Propagation:** This represents the propagation of a gluon, which is the mediator of the strong force. The gluon carries a color charge, represented by the indices A, B , and a momentum q . The covariant propagator is given by the expression:

$$\frac{i\delta^{AB}}{q^2 + i\epsilon} \left(-g^{\mu\nu} + (1 - \lambda) \frac{q^\mu q^\nu}{q^2} \right),$$

where δ^{AB} is the Kronecker delta, $g^{\mu\nu}$ is the metric tensor, q^μ is the four-momentum of the gluon, and λ is the gauge parameter. [12]

- **Quark Propagation:** The quark propagator shows a quark of flavor f with momentum p and mass m , represented by a straight line. The propagator is given by:

$$\frac{i}{\hat{p} - m + i\epsilon},$$

where $\hat{p} = \gamma^\mu p_\mu$, and γ^μ are the gamma matrices in the Dirac equation. [12]

- **Ghost Propagation:** In non-Abelian gauge theories like QCD, Faddeev–Popov ghosts are introduced to maintain unitarity (time evolution follows the Schrödinger equation) and cancel unphysical degrees of freedom and their propagator is similar to a scalar particle:

$$\frac{i\delta^{AB}}{p^2 + i\epsilon},$$

where p is the momentum of the ghost. [12]

Using these lines to depict different particles, you can combine them into diagrams which represent different interaction processes:

- **Gluon-Quark Interaction:** The triple vertex represents the interaction between a quark and a gluon. The quark changes its color charge after emitting or absorbing a gluon. The interaction vertex is given by:

$$-ig_s T_{ij}^A \gamma^\mu,$$

where g_s is the strong coupling constant, T_{ij}^A are the generators of the SU(3) gauge group, and γ^μ is the gamma matrix. [12]

- Three-Gluon Interaction: The three-gluon vertex shows the self-interaction of gluons, a unique feature of non-Abelian gauge theories. The vertex factor is:

$$g_s f^{ABC} [g^{\mu\nu}(p - q)^\rho + g^{\nu\rho}(q - r)^\mu + g^{\rho\mu}(r - p)^\nu],$$

where f^{ABC} are the structure constants of the SU(3) gauge group, and p, q, r are the momenta of the gluons. [12]

- Four-Gluon Interaction: This represents the interaction between four gluons, another characteristic of non-Abelian gauge theories, with the vertex factor [12]:

$$-ig_s^2 [f^{ABE} f^{CDE} (g^{\mu\sigma} g^{\nu\rho} - g^{\mu\rho} g^{\nu\sigma}) + f^{ACE} f^{BDE} (g^{\mu\nu} g^{\sigma\rho} - g^{\mu\rho} g^{\nu\sigma}) + f^{ADE} f^{BCE} (g^{\mu\nu} g^{\rho\sigma} - g^{\mu\sigma} g^{\nu\rho})]$$

In QCD computations, these vertices and propagators are used to calculate scattering amplitudes and cross-sections for processes involving strongly interacting particles, but these processes are specifically extremely relevant in the context of hadronization.

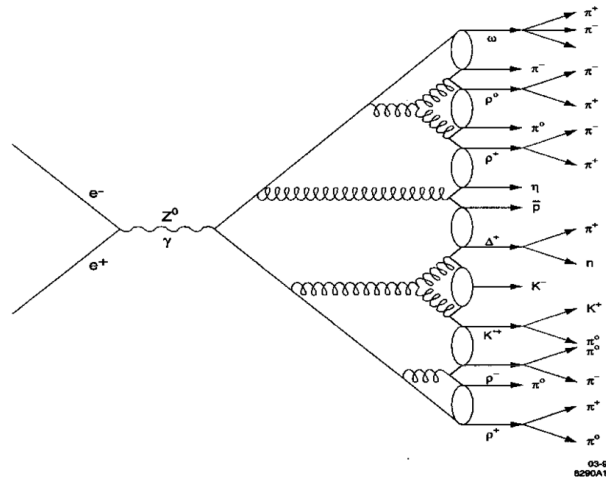


Figure 1.3: Illustration of an example hadronization process in high energy physics.

[16]

Figure 1.3 depicts an instance of hadronization, which is the process through which quarks and gluons transform into hadrons, including protons, neutrons, and mesons. Specifically, the diagram showcases an electron-positron (e^-e^+) annihilation event, where the collision of an electron and a positron at high energies results in the production of a Z^0 boson or a photon (γ). This intermediate boson subsequently decays into pairs of quarks and antiquarks. However, due to the strong force, these quarks do not exist independently for long and begin to undergo hadronization. [16]

Hadronization involves the formation of hadronic jets, as the quarks and gluons move away from each other. During this process, new quark-antiquark pairs are generated from the vacuum, leading to the creation of mesons (such as π and ρ) and baryons (like p and Δ^+). It should also be kept in mind that hadronization is a non-perturbative quantum chromodynamics (QCD) process, as it occurs at low-energy scales where the strong force becomes exceptionally strong, rendering perturbative techniques ineffective.

Deep inelastic scattering (DIS) plays a crucial role in investigating the internal structure of hadrons. In DIS, a high-energy electron or another lepton scatters off a nucleon, causing the expulsion of quarks that subsequently undergo hadronization. By studying the scattering cross-section, valuable information regarding the momentum and spin distribution among the quarks and gluons within the nucleon can be obtained.

Understanding hadronization is also vital in jet formation: In high-energy collisions, quarks and gluons are produced, which then emit additional gluons or quark-antiquark pairs, resulting in the formation of particle jets. Experimentally, the study of hadronization involves observing these jet-like final products in particle accelerators like the Large Hadron Collider (LHC) and comparing the observed data with theoretical models.

1.4.2 Colliders

Colliders, which are sophisticated scientific instruments, are designed to propel subatomic particles to high energies and subsequently collide them. These collisions at high energies provide physicists with the opportunity to investigate the characteristics of these particles in ways that would not be feasible at lower energy levels. As stated by Einstein's equation, $E = mc^2$, the collision energy directly influences the mass, and thus the energy, of the particles that can be generated. There are a few primary types of particle accelerators utilized in collider experiments [17]:

1. **Synchrotron:** A synchrotron functions by accelerating particles along a circular path through a series of magnets that progressively enhance their velocity with each revolution. The increasing strength of the magnetic field ensures that the beam of charged particles remains on a predetermined circular trajectory.
2. **Cyclotron:** The cyclotron represents an earlier version of a circular accelerator, where particles are accelerated outward from the center along a spiral path. This type of accelerator is suitable for achieving moderate kinetic energies in particles.
3. **Linear Accelerator (Linac):** A linear accelerator propels charged particles along a straight path using a series of oscillating electric potentials along its length. As the particles pass through each section of the accelerator, they are sped up incrementally by the electric field, synchronized with the particle beam to increase the particles' velocity.

In the realm of high-energy physics, several colliders stand out [18]:

1. **The Large Hadron Collider (LHC):** This is a synchrotron accelerator that is situated at CERN. This colossal machine currently holds the title of the world's largest and

highest-energy particle collider. It facilitates the collision of protons or heavy ions at energies reaching up to 13 TeV. Notably, the LHC played a pivotal role in the discovery of the Higgs boson, thereby confirming the final missing piece of the Standard Model.

2. **Tevatron:** This is a synchrotron accelerator that was very popular prior to the LHC's reign. Located at Fermilab, it held the distinction of being the highest-energy particle collider. It played a crucial role in the detection of the top quark, one of the fundamental particles within the Standard Model.
3. **Relativistic Heavy Ion Collider (RHIC):** This is a synchrotron accelerator, nestled within the Brookhaven National Laboratory, the RHIC specializes in the collision of heavy ions, such as gold, to recreate the conditions that existed in the early universe mere moments after the Big Bang.
4. **RIKEN Ring Cyclotron:** This is a cyclotron accelerator at RIKEN, and is one of several accelerators intended to propel nuclear physics research forward. With the ability to accelerate heavy ions, it plays a crucial role in the ion accelerator complex, supporting experiments that investigate the characteristics and composition of atomic nuclei.
5. **Superconducting Ring Cyclotron:** This is a cyclotron accelerator that incorporates superconducting magnets, enabling a more streamlined structure and improved acceleration. This state-of-the-art equipment significantly enhances the capabilities of RIKEN's accelerator complex by delivering highly intense ion beams, which are crucial for pioneering research in nuclear physics as well as applications in the fields of medicine and engineering.
6. **88-Inch Cyclotron:** This is a cyclotron accelerator, named after the size of its magnet pole, is a flexible particle accelerator that is predominantly utilized for research

in nuclear science and its associated disciplines. It facilitates a diverse array of scientific inquiries such as generating heavy element isotopes, examining nuclear reaction processes, and advancing novel forms of radiation therapy for treating cancer.

7. Stanford Linear Collider (SLAC): This collider was a linear accelerator at SLAC National Accelerator Laboratory in California, USA, and was famous for its experiments that collided electrons with positrons. It contributed significantly to the understanding of the electroweak interaction.
8. Future colliders: Numerous proposals have emerged for novel colliders that would surpass the LHC in terms of energy or cater to specific research objectives. Some such proposals are the International Linear Collider (ILC) and Future Circular Collider (FCC), which would facilitate electron-electron and electron-positron collisions to enable precision measurements.

Colliders have played a pivotal role in validating the predictions of the Standard Model and have provided invaluable insights into the behavior of matter at high energies. They have allowed scientists to delve into the exploration of symmetries and conservation laws, comprehend the intricate dynamics of the strong force responsible for binding quarks within protons and neutrons, and embark on the quest for new particles that may hold clues about the enigmatic dark matter in our universe.

1.5 Research Objectives

Particle physics investigates the fundamental building blocks of matter and their interactions, providing valuable insights into the origins and structure of the universe. To study these particles, high-energy particle colliders such as the LHC propel particles to speeds close to that of light, resulting in collisions that produce a collimated spray of secondary particles

called jets. The identification and characterization of these jets, commonly known as jet tagging, play a crucial role in the discovery of new particles and the precise measurement of parameters within the Standard Model.

Specifically, jet tagging is essential for distinguishing between signal and background events in particle collision data. Effective tagging methods enhance the sensitivity of searches for new particles and the measurement of properties of known particles. It allows physicists to sift through vast datasets in search of rare processes, making it a fundamental technique for confirming the existence of predicted particles and uncovering unexpected phenomena in physics. The advancement of jet tagging models directly translates to more accurate and efficient data analysis, improving the detection of rare events and minimizing false positives. By utilizing sophisticated algorithms, physicists can analyze data with greater precision, gaining deeper insights into the behavior of fundamental particles and the underlying forces that govern them. The primary objective of this research is to establish the theoretical parameters for the most effective classifier in jet tagging and validate them through model training and optimization. Specifically, the study aims to determine the theoretical upper limits of classifier performance in jet tagging by employing generative AI trained on current data from collider experiments. Subsequently, a machine learning model will be developed to approach these theoretical limits, utilizing state-of-the-art algorithms and optimization techniques. Finally, the performance of the optimized model will be compared against existing benchmarks to assess its effectiveness.

Chapter 2

Machine Learning

2.1 Introduction to Artificial Intelligence and Machine Learning

Artificial Intelligence (AI) and Machine Learning (ML) have taken over as one of the worlds leading industries, but it is often unknown what differences these branches hold.

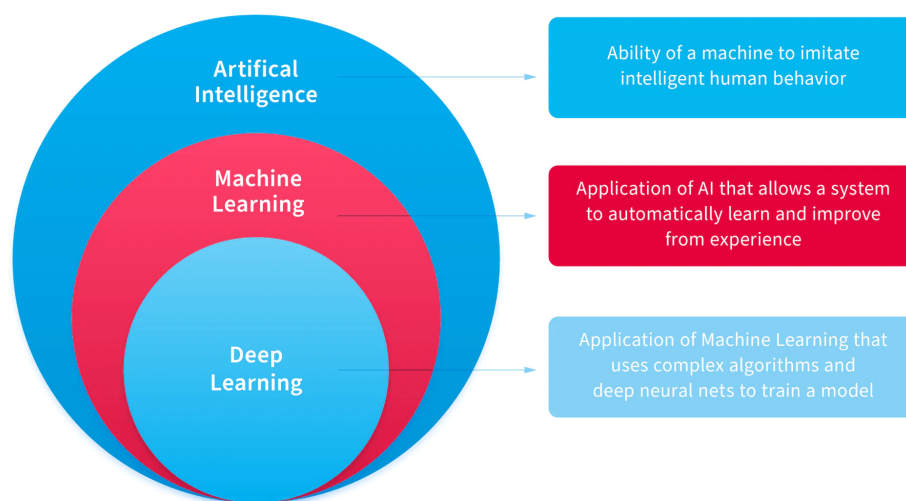


Figure 2.1: Venn Diagram explaining the differences between AI, ML and Deep Learning. [2]

AI is a comprehensive concept that encompasses the notion of machines performing tasks in an intelligent and smart manner. It encompasses a wide range of capabilities, from simple algorithms that can solve specific tasks to complex systems that can reason and learn. The objective of AI is to replicate human cognitive functions, and it can be rule-based or learning-based. In Figure 2.1, AI is depicted as the largest circle, signifying its inclusion of various approaches, including machine learning and deep learning.

ML, on the other hand, is a subset of AI that focuses on the idea that systems can learn from data, identify patterns, and make decisions with minimal human intervention. Unlike traditional AI, ML does not rely on explicit programming to perform tasks; instead, it learns from experience. Machine learning algorithms are designed to enhance their performance in specific tasks over time as they are exposed to more data. Figure 2.1 illustrates ML as a circle within the broader concept of AI, indicating that it is a means through which AI can be achieved.

Finally, Deep Learning (DL) is a further subset of machine learning that involves neural networks with multiple layers, hence the term "deep." These deep neural networks are designed to learn hierarchies of features from data by constructing intricate models capable of handling extensive datasets. Deep learning has played a pivotal role in recent advancements in AI, particularly in areas such as image and speech recognition. In Figure 2.1, DL is represented as the smallest circle within ML, highlighting it as a specific and advanced type of machine learning.

Within the branch of machine learning, we will focus on two main types of models: Discriminative and Generative.

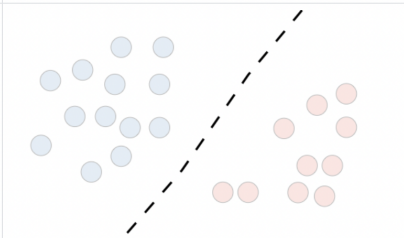
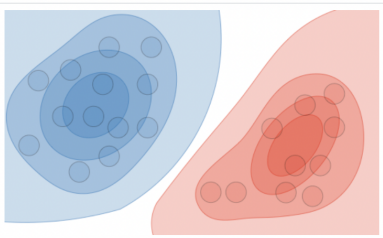
	Discriminative model	Generative model
Goal	Directly estimate $P(y x)$	Estimate $P(x y)$ to then deduce $P(y x)$
What's learned	Decision boundary	Probability distributions of the data
Illustration		

Figure 2.2: Illustration of how Generators and Classifiers are different in their learning methodology. [3]

Discriminative Models focus on determining the decision boundary between classes. They estimate the conditional probability $P(y|x)$, where y is the assigned label for a specific class and x is the features of the input data. The discriminative models predict the probability of the output class given the input features without assuming an underlying data generation model. The decision boundary, which in the left half of Figure 2.2 is depicted as a dashed line, is the outcome of the model learning from labeled data points to predict the label of new, unseen data points.

Generative models focus on learning the distribution of data points within each class through the conditional probability $P(x|y)$, in contrast to discriminative models. By applying Bayes' Theorem, these models can infer $P(y|x)$. The shaded regions in the right half of Figure 2.2 represent the probability distributions of the data points acquired by the model. Moreover, generative models have the ability to create novel synthetic data points that imitate the training data.

By utilizing Bayes' Theorem, the conditional probability can be expressed as:

$$P(y|x) = \frac{P(x|y)P(y)}{P(x)} \quad (2.1)$$

In this equation, $P(x)$ is commonly considered a normalizing factor that guarantees the probabilities sum up to one, and is not explicitly learned by generative models. This expression, as depicted in Eq. 2.1, emphasizes the relationship between discriminative and generative methodologies [3].

where:

- $P(y|x)$ is the probability of class y given features x .
- $P(x|y)$ is the likelihood, or the probability of observing features x assuming it is an instance of class y .
- $P(y)$ is the prior probability of class y .
- $P(x)$ is the overall probability of observing features x .

2.1.1 Modeling

When it comes to modeling on data, there exists several learning methods, but gradient descent is one of the most popular methods to appropriately learn data. Gradient Descent is an optimization algorithm widely used in machine learning to minimize a cost function, represented as $J(\theta)$, by iteratively moving in the direction of the steepest descent as defined by the negative of the gradient. In the context of machine learning, θ represents the model's parameters, and the cost function quantifies the error between the predicted and actual values.

The general update rule for gradient descent is given by [4]:

$$\theta := \theta - \alpha \cdot \nabla_{\theta} J(\theta) \tag{2.2}$$

where:

- θ is a vector of parameters,
- α is the learning rate,
- $\nabla_{\theta}J(\theta)$ is the gradient of the cost function with respect to θ .

There are different types of Gradient Descent algorithms based on the amount of data used to calculate the gradient. Batch Gradient Descent works by computing the gradient using the whole dataset, which is usually computationally expensive and not good for large datasets. Stochastic Gradient Descent (SGD) computes the gradient using a single sample, which is not only less computationally intensive but adds randomness that can help escape local minima. Finally, there is Mini-batch Gradient Descent, which computes the gradient using a mini-batch of samples, almost like the best of both worlds.

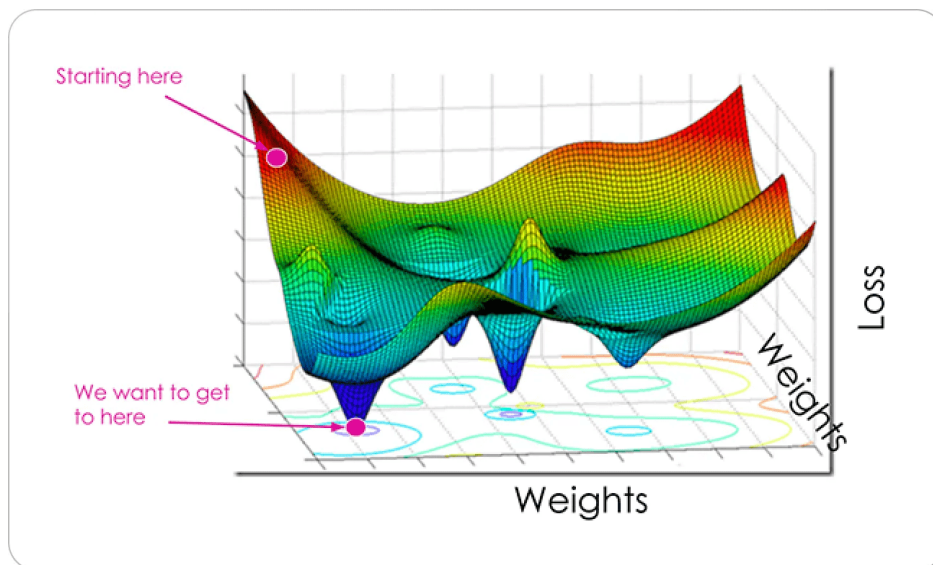


Figure 2.3: Within machine learning, this image depicts how gradient descent works. [4]

Figure 2.3 depicts a visualization of a loss function in a 3-dimensional space where the axes represent two model weights and the loss. The goal of gradient descent is to find the combination of weights that correspond to the lowest point in the plot, marked as "We want to get to here". The point marked "Starting here" signifies the initial parameters from which

the optimization process begins. This surface plot illustrates how gradient descent navigates the terrain of the loss function to find the minimum. With each iteration, the weights are updated in the opposite direction of the gradient of the loss at that point in the parameter space, continuously moving down the slope until the lowest point is reached, representing the optimal solution. There exist several local minima which is why it is important to make sure you don't get stuck there, and this can be done with several methods, but our study focuses on exploiting random subsets of points to introduce noise and escape local minima through stochastic gradient descent. Another important factor to escaping these local minima is proper data preprocessing.

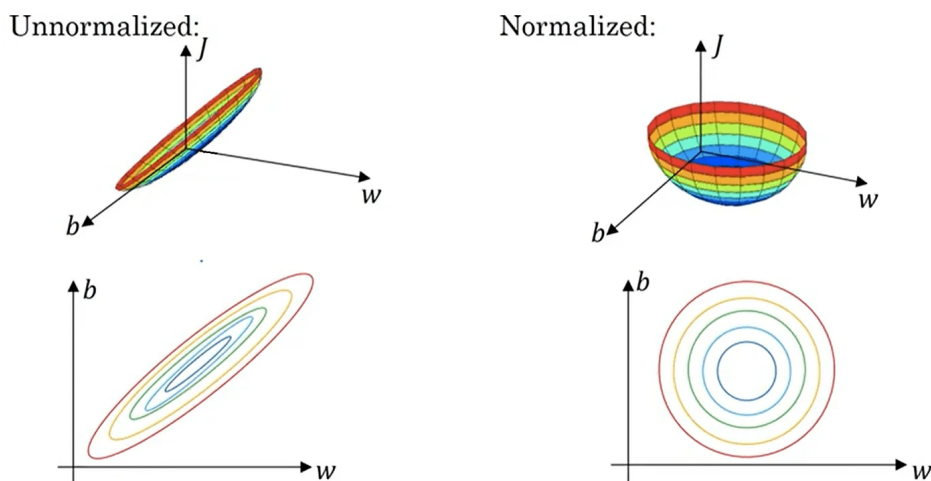


Figure 2.4: Normalization of data. [5]

When it comes to the process of learning data, one of the key features involves the quality of your data, which is why high-quality features must be relevant, informative, and predictive of the desired output. Preprocessing, which includes cleaning and transforming data oftentimes by normalizing, is vital in ensuring that the data is in the best possible format for the model to learn effectively. Feature scaling, such as normalization, is particularly important for models that utilize gradient descent for optimization. Figure 2.4 demonstrates the effect of feature scaling on the optimization landscape. On the left, we see an unnormalized

feature space, where the vastly different scales of features lead to an elongated and inefficient path for gradient descent. On the right, after normalization, the features contribute equally, creating a symmetrical loss surface that allows for a more direct path to the minimum. This illustrates how proper data preprocessing can facilitate more efficient model training especially when using gradient descent.

2.1.2 Classifiers

Machine learning algorithms can be broadly categorized into three main types, based on the nature of the learning feedback available.

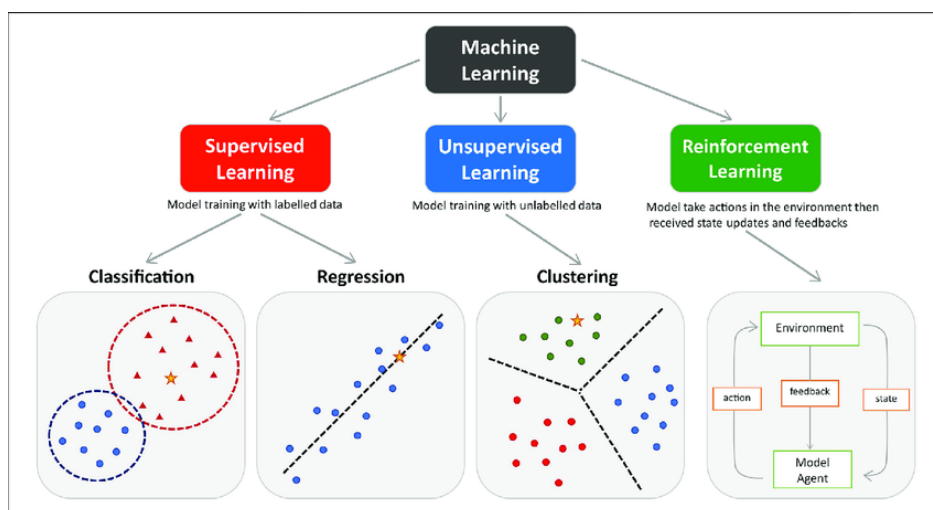


Figure 2.5: The primary methods to approach classification in machine learning. [6]

In supervised learning, the algorithm learns from a labeled dataset, which includes both the input features and the desired output. The aim is to learn a mapping function $h : X \rightarrow Y$ from inputs x to outputs y . Even within supervised learning, we primarily encounter two types of tasks [19]:

- **Classification:** The output variable y is a category and the goal of a classifier is to accurately predict the category for new instances of data based on the learned mapping.

The typical loss function used to evaluate classifier performance is cross-entropy loss, given for binary classification by:

$$L(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})], \quad (2.3)$$

where y is the true label, and $\hat{y}$ is the predicted probability of the positive class.

- Regression: The output variable y is a real value, which represents the magnitude of a measurement or even some kind of count.

Moving forward, there is unsupervised learning and this type involves models that learn patterns from unlabeled data. The most common task under this category is clustering, where the goal is to group the input data into clusters based on feature similarity without predefined labels.

Finally, there is reinforcement learning where an agent interacts with a dynamic environment to achieve a certain goal. The agent has a decision function that allows it to follow through on an action and enter a new state, then it learns from the consequences of its actions, through rewards or penalties, to make a sequence of decisions.

Throughout our research we will focus on supervised learning, specifically classifying events through a "decision boundary," although one of the biggest issues we face in this is properly fitting the data.

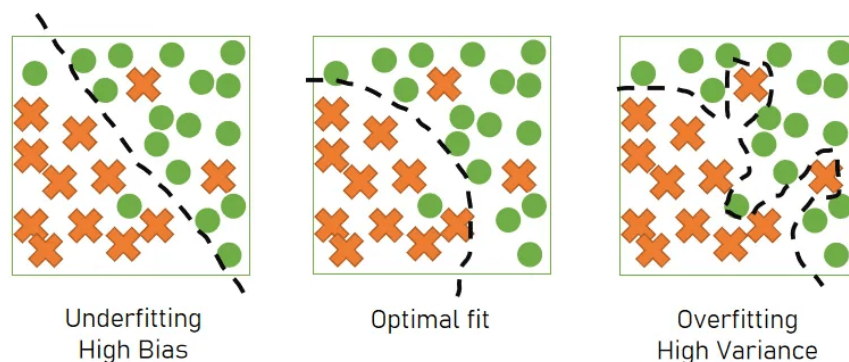


Figure 2.6: Overfitting and underfitting in the context of classification. [7]

Figure 2.6 illustrates three scenarios of model fitting in machine learning: underfitting, optimal fit, and overfitting, which are pivotal in understanding the model's performance.

The initial panel on the left, denoted as "Underfitting High Bias," exhibits a model that possesses an excessively simplistic decision boundary. Consequently, it fails to accurately capture the genuine structure and distribution of the data. Such models exhibit a substantial error rate on both the training and testing datasets, primarily due to the inherent high bias ((bias refers to the error introduced by approximating a real-world problem) resulting from their simplicity. The middle panel represents an "Optimal fit," wherein the model's decision boundary effectively distinguishes between the two classes. This model strikes a harmonious balance between bias and variance (variance refers to the error due to the model's sensitivity to small fluctuations in the training set), as it fits the training data well while also generalizing effectively to unseen data. Lastly, the rightmost panel, labeled as "Overfitting High Variance," showcases a model with an intricate decision boundary that accommodates the noise and outliers present in the training data. Consequently, this leads to a low training error but poor generalization to new data, as indicated by the potential for high testing error due to the model's high variance.

An essential goal in machine learning is to find the right balance between bias and variance

to achieve the best generalization performance. Models with high bias tend to underfit the data, while models with high variance tend to overfit.

2.1.3 Generators

Generative models are a fundamental aspect of machine learning, where the objective is to understand and replicate the distribution of data. These models are not only capable of predicting outcomes but also generating new data instances that are similar to the observed ones.

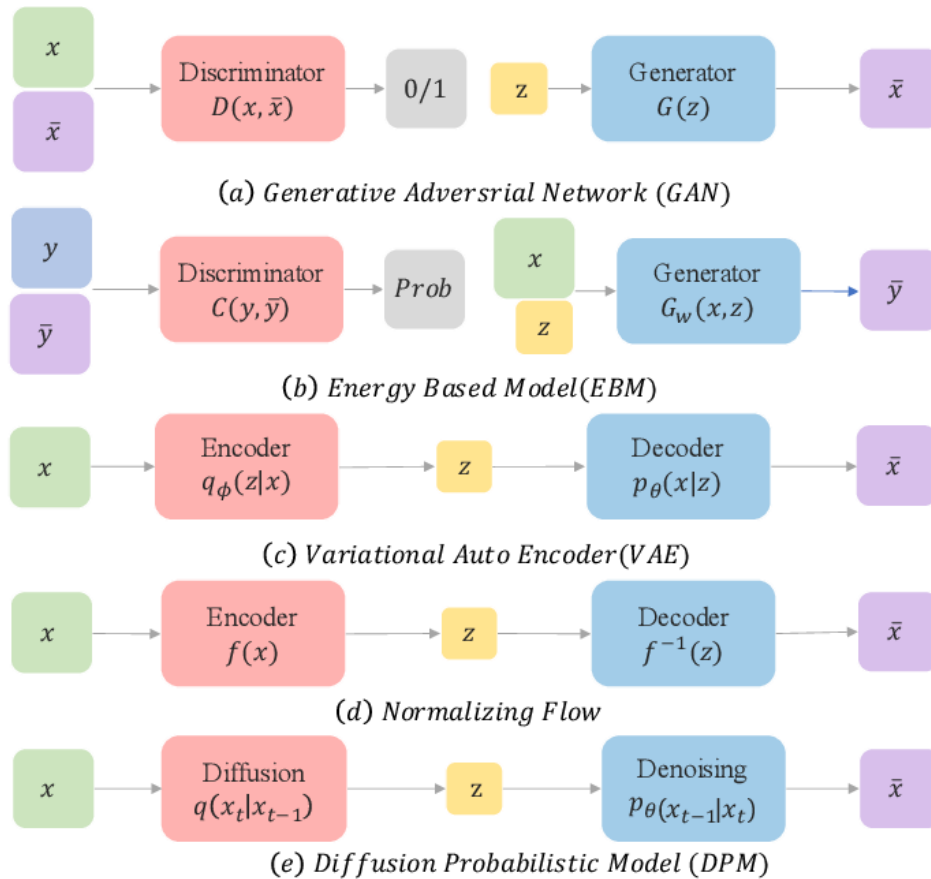


Figure 2.7: An overview of all the primary kinds of generative models used today. [8]

Figure 2.7 introduces some of the most common generative models [8]:

1. Generative Adversarial Network (GAN): A GAN is comprised of a pair of neural networks, namely the generator G and the discriminator D , which undergo training concurrently using adversarial techniques. The primary objective of the generator G is to produce synthetic data that closely resembles real data, whereas the discriminator D aims to differentiate between authentic data x and artificial data $\tilde{x}$ generated by G . The dynamics of a GAN can be elucidated through a min-max game framework:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \quad (2.4)$$

where p_{data} is the data distribution, p_z is the prior on input noise variables, and z is a source of noise.

2. Energy Based Model (EBM): EBMs are designed to learn an energy function $E(x)$ that assigns low values to probable configurations of the input variables and high values to improbable ones. The probability distribution is established based on this energy function:

$$p(x) = \frac{\exp(-E(x))}{Z} \quad (2.5)$$

where Z is the partition function that normalizes the probabilities. This format falls very closely in line with Boltzmann distributions in statistical mechanics.

3. Variational Autoencoder (VAE): VAEs are composed of two main components: an encoder $q_\phi(z|x)$ and a decoder $p_\theta(x|z)$. The primary objective of VAEs is to enhance the likelihood of generating correct data by acquiring knowledge from a latent variable model. The encoder is responsible for approximating the posterior distribution by moving forward, while the decoder is responsible for modeling the conditional likelihood

to transform back into original data, and the likelihood is depicted as:

$$p_{\theta}(x) = \int p_{\theta}(x|z)p(z)dz \quad (2.6)$$

where θ are the parameters of the decoder. VAEs optimize the variational lower bound on the likelihood.

4. Normalizing Flow: Normalizing flows are utilized to convert a basic distribution, such as a Gaussian distribution, into a more intricate one through a series of invertible transformations. These models offer precise log-likelihoods, and the change of variables formula is applied in the process:

$$p_x(x) = p_z(z) \left| \det \left(\frac{\partial f^{-1}}{\partial x} \right) \right| \quad (2.7)$$

where f is the flow transformation, f^{-1} its inverse, $p_z(z)$ the distribution of the latent variable z , and $p_x(x)$ the resulting distribution after applying f .

These are all models that are used very often in Machine Learning, but the model that we will be using is the last one, also known as a diffusion model. Diffusion models are an emerging class of generative models that learn to transform data into noise and subsequently reverse this process to generate data from noise. These models operate on the principle of Stochastic Differential Equations (SDEs).

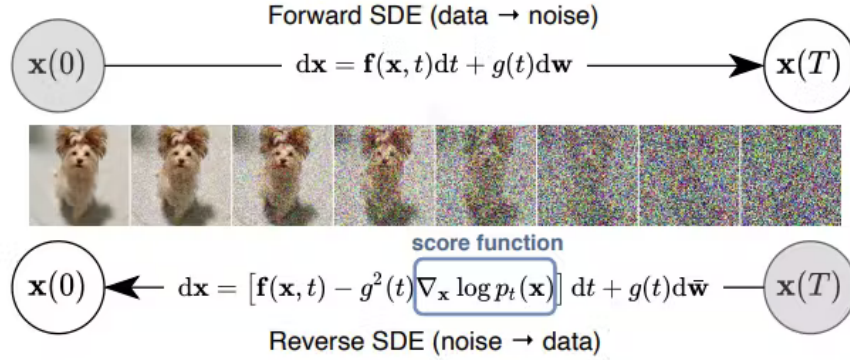


Figure 2.8: Specific depiction of the Diffusion Model used in this research. [9]

Figure 2.8 illustrates this transformative process through two stages: forward diffusion and reverse diffusion.

Forward SDE (Data to Noise): During the initial diffusion phase, we start with the original data $x(0)$ and gradually introduce noise over a sequence of time intervals. This results in a state $x(T)$ at time T , which should exhibit characteristics of pure noise or entropy if given enough time to evolve. The forward stochastic differential equation (SDE) can be expressed as:

$$dx = f(x, t) dt + g(t) d\omega, \quad (2.8)$$

where $f(x, t)$ represents the drift coefficient that influences the deterministic evolution of the data, $g(t)$ is the diffusion coefficient that governs the scale of noise, and $d\omega$ denotes the stochastic differential that captures the increments of a Wiener process. [9]

Reverse SDE (Noise to Data): On the contrary, the inverse diffusion process aims to reconstruct the initial data from the noise. The ability to generate is evident during this stage, where the inverse SDE utilizes a score function, which is the gradient of the logarithm of the probability with respect to the data, to direct the transformation from noise back to

data:

$$dx = [f(x, t) - g(t)^2 \nabla_x \log p_t(x)] dt + g(t) d\tilde{\omega}, \quad (2.9)$$

where the score function $\nabla_x \log p_t(x)$ acts as a guiding mechanism from randomness to structured data, and $d\tilde{\omega}$ represents the reverse process noise term. [9]

The series of images in the upper row of Figure 2.8 illustrates the data becoming progressively obscured by noise as time passes, transitioning from $x(0)$ to an unclear state $x(T)$. The lower row showcases the reverse diffusion, starting from the noisy state $x(T)$ and gradually revealing structure to recover the original data $x(0)$.

Diffusion models have played a crucial role in producing high-quality samples and will be particularly useful for generating jet data, which we will further elucidate in Section 2.4.

2.2 Deep Learning

Neural networks are a cornerstone of machine learning and come in various architectures designed for specific tasks, and some of the most relevant ones are as follows [20]:

1. Feedforward Neural Networks (FNNs), also referred to as Multilayer Perceptrons (MLPs), are a fundamental type of artificial neural network architecture where information flows in a unidirectional manner from input nodes, through hidden nodes, to output nodes, without cycles or loops.
2. Convolutional Neural Networks (CNNs) are designed for processing data with a grid-like topology, such as images. They utilize convolutional filters to generate feature maps that capture spatial characteristics like edges, textures, and colors, which are further abstracted by subsequent layers for tasks like classification.
3. Recurrent Neural Networks (RNNs) are tailored to process sequential data, with connections between nodes forming a directed graph along a sequence. This structure

allows RNNs to utilize internal memory to process input sequences, with applications in areas like natural language processing and time-series prediction.

4. Long Short-Term Memory Networks (LSTMs) are a specialized type of RNNs adept at learning long-term dependencies. They use gates to regulate the flow of information, allowing them to retain information over lengthy sequences, making them effective for tasks with long-term dependencies.
5. Graph Neural Networks (GNNs) operate on graph-structured data, leveraging the relationships between graph-represented data points. Through node relations, GNNs capture graph-like dependencies, effectively treating each point as a node in a broader graph, which makes them suitable for tasks such as social network analysis, where for example a person may be a node and has lines connecting to each of their friends.

These architectures are often combined to get the best of multiple worlds, but another very popular architecture is the addition of a transformer. Transformers are a type of architecture that relies on self-attention mechanisms to weigh the influence of different parts of the input data. The multihead attention mechanism allows the model to focus on different positions of the input sequence, making them effective for a variety of sequential tasks, and very good for jet tagging which can consist of up to 100 particles, each with up to 5 associated features per event.

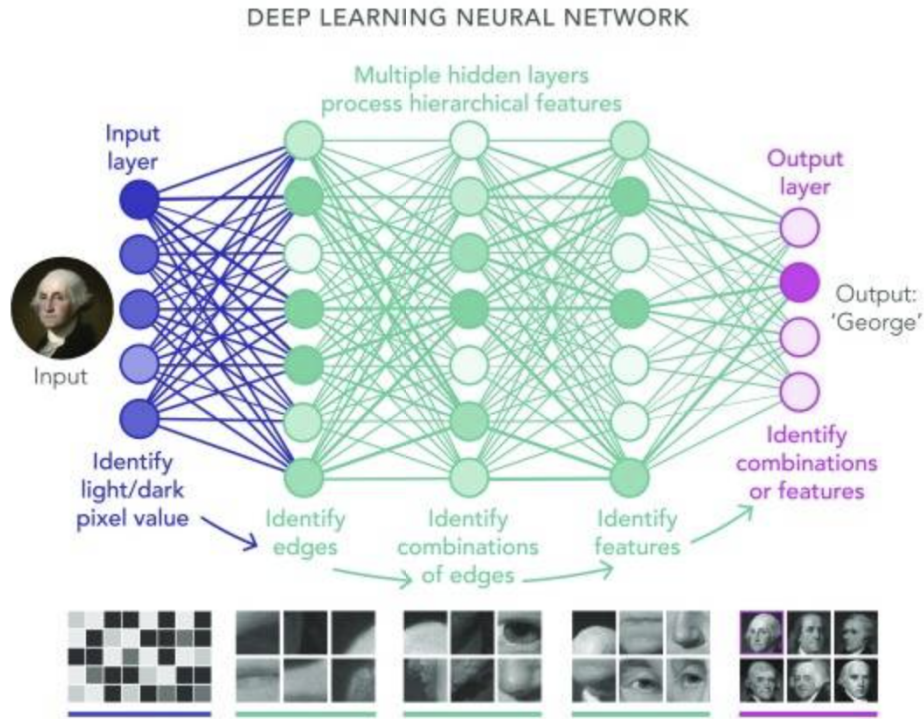


Figure 2.9: Specific depiction of the Diffusion Model used in this research. [10]

Figure 2.9 illustrates a basic deep learning neural network architecture. First we have an input layer which represents the initial data fed into the network. In the case of image processing, this would be the raw pixel data. Next, we have hidden layers and these are layers between the input and output layers where feature extraction occurs. Each subsequent hidden layer typically transforms the features from the previous layer to increasingly abstract representations. Finally, we have the output layer that provides the output of the network. For classification tasks, this would typically involve a softmax function that gives a probability distribution over the target classes. In the context of Figure 2.9, the network seems to process an image of George Washington and outputs the name "George." It simplifies the complexity of the neural network architecture by showing how the network could identify low-level features like edges in the first hidden layer, combinations of edges in the next, and more complex features such as parts of a face in subsequent layers, before finally

arriving at a high-level representation that corresponds to a class label.

2.3 Tools and Frameworks

Developing machine learning models, like Graph Neural Networks (GNNs), requires a range of tools and frameworks to aid in coding, training, and deployment. The primary libraries utilized in this process are Tensorflow and PyTorch. Tensorflow, an open-source platform developed by Google, is widely recognized for its exceptional capabilities in numerical computation and machine learning deployment across various platforms. On the other hand, PyTorch is Facebook's AI Research lab's contribution to the open-source community, providing a dynamic computation graph that facilitates rapid and flexible model prototyping. In addition to these libraries, there are other significant libraries available for machine learning, such as scikit-learn, keras, and horovod. Scikit-learn is renowned for its simplicity, offering a variety of algorithms for classification, regression, and clustering, although it is not specifically designed for deep learning. Conversely, keras serves as a neural network API for TensorFlow, with a focus on enabling quick experimentation. Lastly, horovod is a distributed training framework that enhances the efficiency and speed of training deep learning models. [21]

One of the major challenges faced in large-scale machine learning projects is computability. GPU capabilities play a crucial role in efficiently training deep neural networks, but they also present their own challenges, including memory limitations and computational bottlenecks. In general, machine learning engineers must navigate the learning curves associated with different tools, ensure compatibility between them, and constantly stay updated with the rapidly evolving landscape of machine learning technologies.

2.4 Research Applications

Neural network-based classifiers and generative models play a crucial role in the field of High Energy Physics (HEP) by enhancing the accuracy of data analysis studies. Through the utilization of deep learning (DL), classifiers are able to learn intricate features from data that are not easily captured by traditional methods. This is particularly significant in the context of jet tagging, where distinguishing between a jet from a top quark and a gluon can be subtle. Moreover, DL models have the capability to automatically extract pertinent features from raw data, such as energy deposits in a detector, without the need for manual feature engineering. This empowers classifiers to leverage all available information, potentially uncovering new patterns. The adaptability of DL classifiers allows them to easily adjust and improve with the incorporation of new data.

On the other hand, generative models have the ability to generate synthetic data that closely resembles real experimental data. This proves to be highly advantageous in scenarios where certain types of jets, particularly those originating from rare processes, are under-represented in the available datasets. By training classifiers on synthetic data, they can effectively recognize these rare jet types. Additionally, generative models can be trained to model the distribution of "normal" jets. When a jet deviates significantly from the model, it can be identified as an anomaly. This is particularly useful in the search for new particles or interactions, as unexpected jet types may indicate the presence of new physics. Furthermore, traditional simulations in HEP are computationally intensive due to the detailed modeling of particle interactions and detector responses. Generative models offer a faster alternative by emulating these simulations, thereby accelerating the research cycle. By generating multiple plausible outputs, generative models can also assist in estimating the uncertainties inherent in predictions, which is crucial for assessing the reliability of conclusions drawn in experimental physics.

Chapter 3

Jet Tagging

3.1 Design and Implementation

The graph in Figure 3.1 shows numerous Receiver Operating Characteristic (ROC) curves from different studies, all of which have been slowly improving upon one another, but there seems to be no end. Thus, our research aims to exploit Generative AI to determine the theoretical upper bound for classifier performance. In order to determine optimal classifier performance, we want a classifier to minimize False Positives and maximize True Positives. To determine this value, we can use the Neyman-Pearson Lemma, which is the "uniformly most powerful" measure of whether your data supports one of two hypotheses, and in turn we get the following result for the likelihood ratio (the proof for which can be found in the appendix, section A.1):

$$\mathcal{L}(x) = \frac{p(x|\theta_0)}{p(x|\theta_1)} \quad (3.1)$$

where x represents the data, θ_0 denotes the parameters of the model under the null hypothesis, θ_1 denotes the parameters of the model under the alternative hypothesis, $p(x|\theta_0)$ is the likelihood of observing the data x given that the null hypothesis is true and $p(x|\theta_1)$ is the

likelihood of observing the data x given that the alternative hypothesis is true [22]. We will use this formula to comparatively measure the likelihoods of two hypotheses.

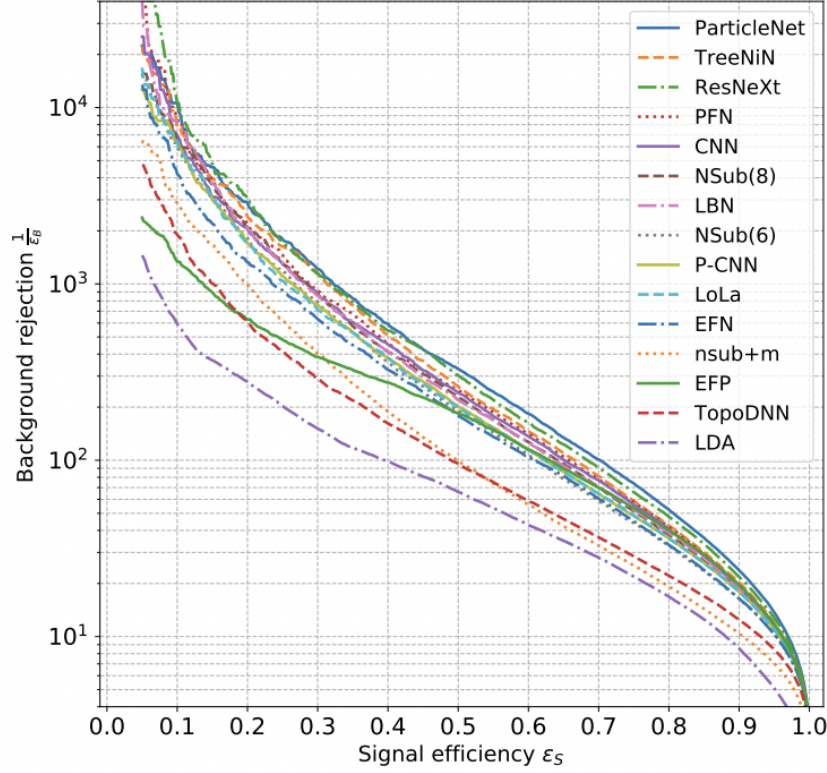


Figure 3.1: ROC curves for numerous high-performing jet tagging models. [11]

The main problem we face is that we currently have raw data, and thus we do not have access to a probability density distribution for the data, and this is where we will use Generative AI. Specifically, Figure 2.8 shows the method for generation of data, and within that image we see that there is a score function, denoted as the gradient of the log of the probability density distribution. Knowing this, our process will go as follows: by creating distinct diffusion generative models for top quark and gluon jets, we can train these models using real collider data (there exists around 2M total open-source top and gluon jet events on Zenodo[23]), with the main goal being to produce synthetic data that

closely resembles the actual jet data when analyzed statistically, with a minimal margin of error. If the generated data aligns successfully with the real data, it suggests that the trained score function effectively represents the true data distribution. Upon solving the differential equation inherent in the diffusion model, we can derive the log-likelihood of the data's probability density. The differences of log-likelihoods between our two models results in the likelihood ratio, as follows:

$$\log \mathcal{L}(x) = \log p(x|\theta_0) - \log p(x|\theta_1) \quad (3.2)$$

This likelihood ratio, having been derived from Neyman-Pearson's Lemma, will have minimized false positives and maximized true positives, and thus representing this ratio as a Receiver Operating Characteristic (ROC) curve enables us to determine optimal classifier performance. In order to classify jets, we will utilize our knowledge of classifiers to develop a Graph Neural Network (GNN) classifier. The effectiveness of this classifier will be measured through ROC curves, which after getting preliminary results, we will continuously enhance to mimic the performance of the theoretical optimal classifier as closely as possible.

3.2 Results

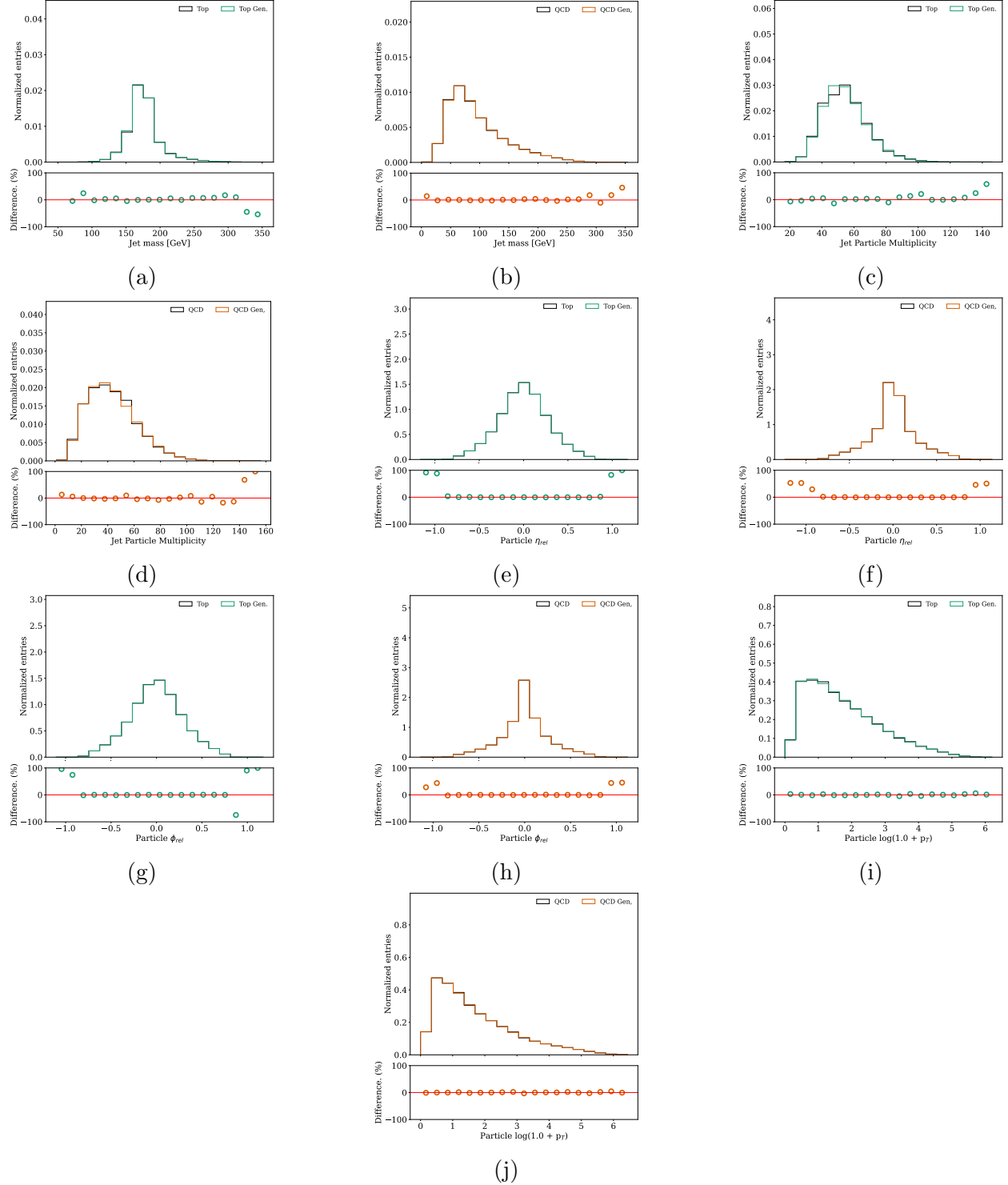


Figure 3.2: Plots of Generated vs. Real Particle and Jet Features for both Tops and Gluons, created by Dr. Vinicius Mikuni.

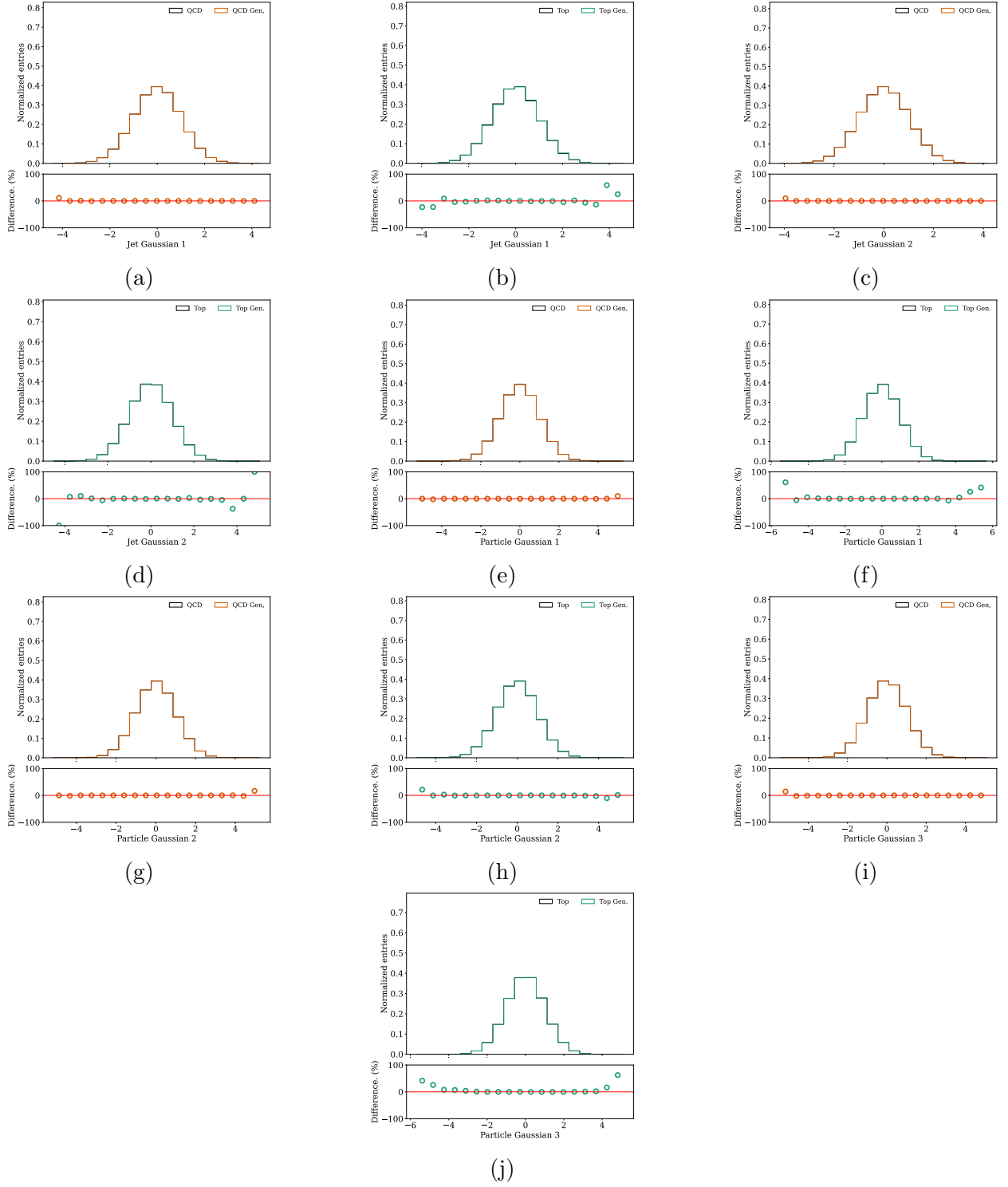


Figure 3.3: Plots of reconstructed Gaussian's for comparisons of the forward and backwards transformation of the diffusion model, created by Dr. Vinicius Mikuni.

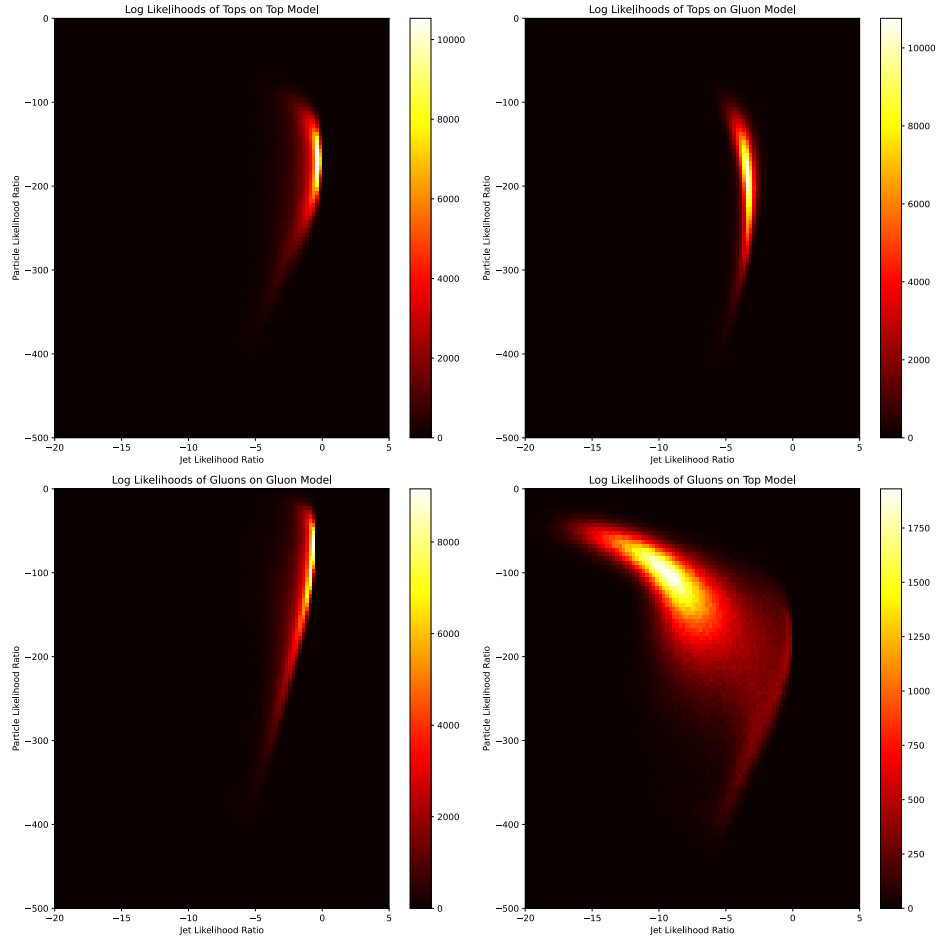


Figure 3.4: These are heatmaps depicting the two model probabilities with the two different datasets across the particles and jets.

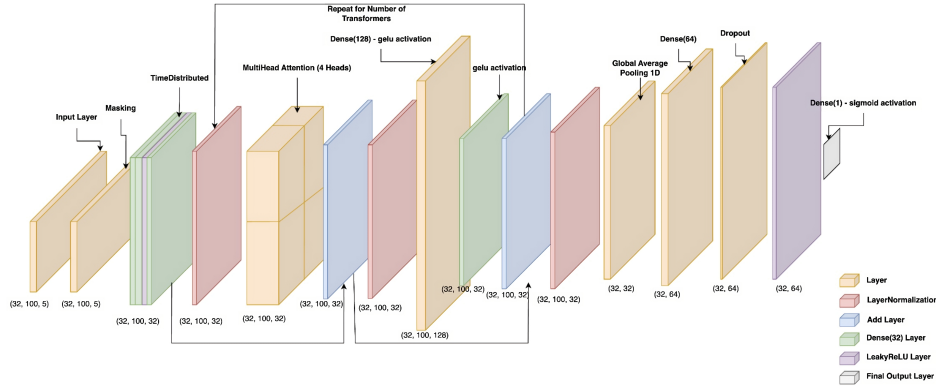


Figure 3.5: This image depicts the classifier model architecture, the code for which can be found in the Code Appendix B.1.

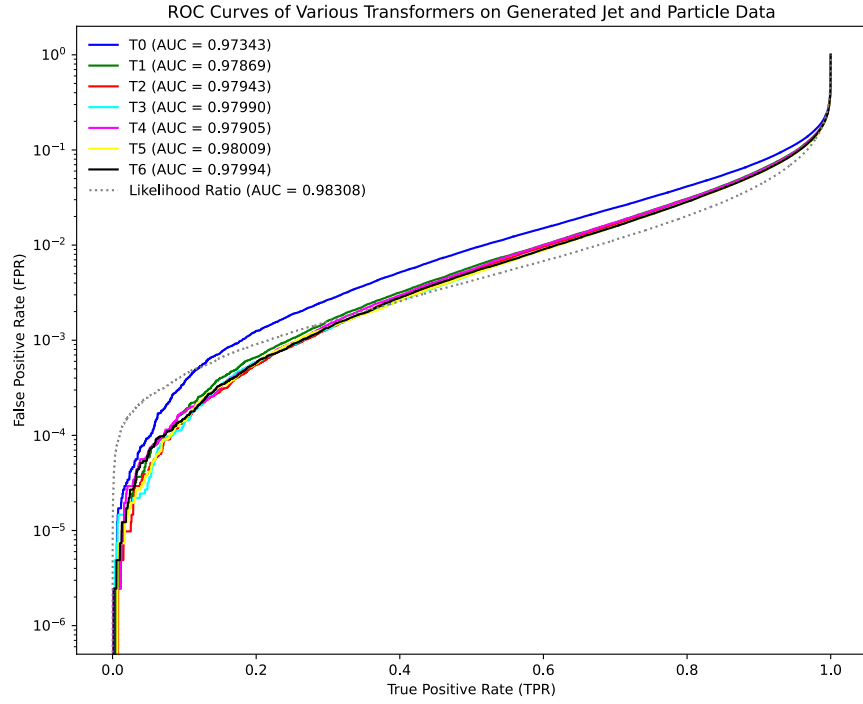


Figure 3.6: ROC curves for generated data with varying numbers of transformers.

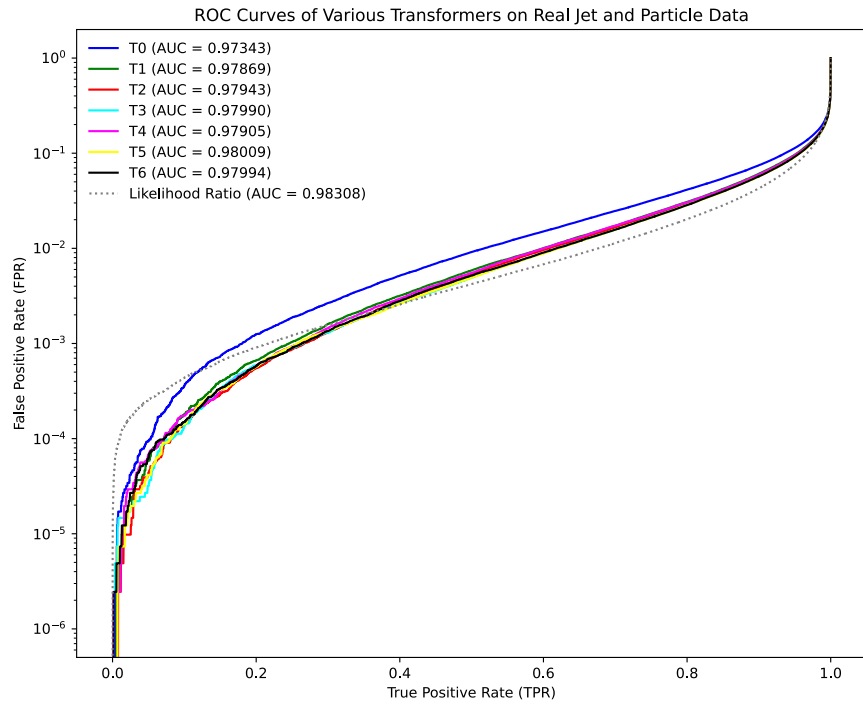


Figure 3.7: ROC curves for real data with varying numbers of transformers.

3.3 Discussion

3.3.1 Diffusion Model Certainty

In order to ensure the accuracy of our 2 Diffusion Generative Models (one for Top data and one for Gluon data), as discussed in Section 3.1, the first step is to conduct a test. This test involves coding the model to generate data with an error bound of 1×10^{-5} , as specified in the Code Appendix B.2. By doing so, we anticipate that the generated data will closely resemble our actual data. Upon analyzing Figure 3.2, we observe multiple plots depicting various jet features. Plots (a) and (b) represent the Jet Mass distributions in GeV for Tops and Gluons respectively. From these distributions, it is evident that the real and generated data exhibit minimal differences, indicating a high level of similarity.

Moving forward, plots (c) and (d) illustrate the jet particle multiplicity, which refers to the total number of particles comprising a jet. Upon examination, we find that the generated particle multiplicity aligns closely with the real particle multiplicity on average.

Plots (e) and (f) focus on the particle pseudorapidity, which serves as a spatial parameter characterizing the angle of a particle in relation to the beam axis. Analyzing these plots, we observe that the distribution is almost perfectly modeled, with the exception of outliers at the extreme ends, which is to be expected.

Shifting our attention to plots (g) and (h), these represent the azimuthal angle, which denotes the angle around the beam axis. This parameter, in conjunction with the pseudorapidity, enables the reconstruction of the precise direction of the particle. Upon examining these distributions, we once again find that the generated data closely resembles the real data for both Tops and Gluons.

Finally, plots (i) and (j) depict the particle log momentum, which typically follows a falling distribution. Our generated data effectively captures this pattern, indicating that all of our particle and jet features are accurately represented for both tops and gluons. In

conclusion, our generated data demonstrates a strong resemblance to the real data across all particle and jet features. This confirms the effectiveness of our Diffusion Generative Model in accurately modeling the desired characteristics.

Further validation can be accomplished by comparing the reconstructed Gaussian distributions of both actual and generated data. As explained in Section 2.1.3, generative models operate by transitioning from the initial data to a Gaussian noise distribution and then reversing this process to generate new data from Gaussian noise. If the generated data accurately represents the actual data, we would anticipate a nearly perfect overlap between the reconstructed Gaussian noise from both sources. Upon examining the Gaussian plots for all five features, we observe a remarkable alignment in each case. This strong correspondence strongly indicates that the model is highly proficient in capturing the underlying characteristics of the actual jet data.

3.3.2 Likelihoods

After verifying the accuracy of our diffusion models in representing the actual data, we proceed to solve the score function ODE in order to obtain the logarithm of the probability density distribution. This particular step is crucial as it allows us to generate likelihoods from our synthetic data. In order to calculate these likelihoods, we assess all possible combinations within our probability density distribution across the dataset. By utilizing our model that has been trained on gluon data, we are able to determine the likelihoods for both gluon and top datasets. We anticipate that the gluon model will assign high probabilities to data that truly represents gluons, indicating that the likelihoods for the gluon dataset should be centered around a probability of 1. Conversely, the likelihood of identifying top data as gluon should be significantly low, ideally centered around 0. A similar rationale applies to the top model, where top data should predominantly exhibit probabilities around 1, while

gluon data would tend towards 0.

Referring to Figure 3.4, the two plots on the left—Top data on the Top model and Gluon data on the Gluon model—demonstrate this expectation. In these plots, the y-axis represents particle likelihoods, while the x-axis represents jet likelihoods. Considering that we are discussing log-likelihoods, a distribution centered around 0 corresponds to a likelihood of $e^0 = 1$. This alignment is clearly observed in our analysis of the jet likelihoods of these distributions, thereby confirming our expectations for jet data. It is important to acknowledge that we may not anticipate the same level of precision from the particle data due to its inherent characteristics such as directionality and momenta. While these features play a significant role in distinguishing top data from gluon data, the jet mass (which is included in the jet information) serves as a more dominant feature in this differentiation.

Upon analyzing the right portion of the graph, we can observe that the distributions for the Top data on the Gluon model and Gluon data on the Top model are concentrated around -5 and -10, respectively. These numerical values, which correspond to e^{-5} and e^{-10} , gradually approach zero. This alignment with our anticipated low likelihood for inaccurate classifications further reinforces our findings. This examination serves as a confirmation that our probability distributions excel in accurately representing and analyzing our data. It provides robust validation for the efficacy of our diffusion models in statistical representation and analysis.

As previously mentioned, the log-likelihoods can be employed in combination with Neyman-Pearson's Lemma, as specified in Equation 3.2, to calculate a ratio of likelihoods between our two models on synthetic data. Since these probability distributions accurately represent our "raw" data, this likelihood ratio serves as a critical measure, indicating the optimal achievable classifier performance by maximizing the True Positive Rate (TPR) and minimizing the False Positive Rate (FPR). The most efficient method to visualize and evaluate this function is by utilizing a Receiver Operating Characteristic (ROC) curve. This graphical representa-

tion effectively contrasts the FPR (y-axis) with the TPR (x-axis). Upon examining Figures 3.6 and 3.7, we can observe the likelihood ratio illustrated by the gray dotted line. This depiction ideally corresponds to the best attainable classifier performance.

3.3.3 Classifiers

Upon formulating the optimal classifier, our subsequent goal is to construct a model that not only approximates but ideally replicates the optimal Receiver Operating Characteristic (ROC) curve. We commence with fundamental neural network structures and gradually incorporate more advanced architectures, culminating in a sophisticated transformer model with multi-head attention mechanisms at a high level, as illustrated in Figure 3.5.

The devised architecture is specifically crafted to tackle the intricate, high-dimensional data associated with hadronic jet tagging, encompassing several crucial components:

- **Masking Layer:** Crucial for ensuring that irrelevant or padding data does not impact the model, a critical aspect for managing particle jets of varying lengths.
- **TimeDistributed Layers:** These layers apply operations to each temporal slice of an input, essential for handling the data of each particle independently before aggregation, reflecting the individual contributions of particles to the overall characteristics of a jet.
- **LeakyReLU Layers:** These layers play a significant role in maintaining slight neuron activity, aiding in learning fine details within non-linear and complex patterns. This functionality helps sustain learning even when certain nodes start to produce negative outputs.
- **Add Layer:** Introduces a "skip connection" that helps mitigate the vanishing gradient issue by enabling specific layers to bypass others if they do not contribute to loss reduction.

- **MultiHead Attention:** Featuring four heads, this mechanism enables the network to simultaneously focus on different segments of the particle sequence, capturing intricate inter-particle relationships.
- **Layer Normalization and GELU Activation:** Normalization enhances the stability of the learning process, while GELU activation aids the model in capturing the non-linear relationships inherent in jet data.
- **Global Average Pooling 1D:** This layer aggregates information from all particles, reducing the dimensions of the features while retaining important data.
- **Dense and Output Layers:** These layers, combined with dropout to prevent overfitting, culminate in a single-node output that uses a sigmoid function for binary classification.

In Figure 3.5, there is an arrow pointing back from the Add Layer to the LayerNorm, indicating the recursive nature of transformers in the architecture. This recursive nature can be adjusted to optimize the performance of the model, and training these we achieve numerous AUC scores. The AUC metric stands for Area Under Curve and is important as the higher the value, the stronger your classifier performance, where if the classifier AUC is 1, it can perfectly distinguish events.

Table 3.1: AUC values for various transformers on generated and real jet and particle data

Model	AUC (Generated Data)	AUC (Real Data)
T0	0.97453	0.97343
T1	0.97757	0.97869
T2	0.97795	0.97743
T3	0.97813	0.97900
T4	0.97821	0.97890
T5	0.97883	0.97894
T6	0.97816	0.97816
Likelihood Ratio	0.98308	

The performance assessments shown in Table 3.1 demonstrate that the initial classifier without any transformers (T0) does not achieve the best possible performance. However, the introduction of a single transformer (T1) significantly improves the performance. Further increments in the number of transformers, up to six (T3-T6), result in consistent small improvements for real data and generated data with notable slight decreases at T2, T4 and T6 for generated data, but no substantial changes at the order of 1×10^{-2} or higher. This suggests that beyond a single transformer, additional complexity does not proportionally enhance the performance. Nevertheless, the classifiers are highly effective in distinguishing between tops and gluons. Examining the ROC Curves, we see that the classifier performance is outperformed by the likelihood ratio for TPR greater than 0.3 which is exactly what we expect, and by increasing the number of transformers, we see a similar effect as we saw with the AUC, where we approach the likelihood ratio but are still not there yet.

3.4 Future Research

An important observation regarding the performance of the model, particularly in the region of low True Positive Rate (TPR), requires attention. Theoretically, the Likelihood Ratio is expected to be the optimal classifier, minimizing the False Positive Rate (FPR) across all levels of TPR. However, we notice an anomaly in that our classifier outperforms the Likelihood Ratio at lower TPR values, specifically at or below 0.3. This unexpected deviation, which should not theoretically occur, suggests potential inaccuracies in the Likelihood Ratio. These inaccuracies could arise from small errors in the model calculations, even at the magnitude of 1×10^{-5} , particularly during the computation of likelihoods from synthetic data.

To address this issue, we are considering a few corrective measures. First, we plan on reducing particle count from 100 to 50 in efforts to improve the accuracy of likelihood cal-

culations. This reduction is also expected to expedite the generation of likelihoods, which is currently limited by the computational burden on the ODE solver. We also intend to explore alternative models for generating likelihoods, which may provide more precise predictions and better overall performance. These strategies represent crucial areas for further research and development, which we plan to pursue in the near future.

Conclusion

The conclusion of this thesis presents a comprehensive investigation into the effectiveness of diffusion generative models in the field of high-energy physics, specifically in the context of jet tagging. Our research has successfully demonstrated the ability of these models to generate synthetic data that closely resembles real jet data with remarkable accuracy. This synthetic data generation is particularly important in situations where the actual probability density distributions are not readily available, as it enables the computation of log-likelihoods that are essential for the precise classification of jet particles. By utilizing the Neyman-Pearson Lemma to calculate the likelihood ratio, we have established a robust framework for evaluating classifier performances. This approach has allowed us to quantitatively assess the alignment between the synthetic data and real data, which is confirmed by ROC curves showing model performance below that of the likelihood ratio for TPR above 0.3 as seen in Figures 3.6 and 3.7.

One of the key findings from our research is the high level of similarity in the statistical features of the generated data compared to real data. This is evident from our 0 average residual plots from Figures 3.2 and 3.3. Furthermore, the close-to-zero log-likelihood values in the heatmap from Figure 3.4 when the models correctly identified their associated data type (gluon or top), and the significantly negative values when the models were applied across different categories confirmed the discriminative power of the employed models. Furthermore, the incorporation of graph neural networks has played a crucial role in enhancing

our modeling capabilities, resulting in notable improvements in classifier performance. The refined classifiers as seen in Figure 3.5 have exhibited a substantial boost in performance from the addition of one transformer, and variance smaller boosts and losses with the addition of more, as indicated by the Area Under Curve (AUC) metrics in Table 3.1, highlighting the potential of advanced machine learning techniques in the field of high-energy physics.

Future research should prioritize investigating the improvement of these models, with a specific focus on addressing the anomalies observed in the performance at lower True Positive Rates (TPR). It is suggested that adjustments be made to the modeling approach, such as reducing the particle count or exploring alternative models for generating likelihoods, as these modifications have the potential to enhance the precision and efficiency of these generative models. To summarize, this thesis not only contributes to our understanding of the practical applications of diffusion models in jet tagging but also paves the way for future advancements in the field of high-energy physics. The methodologies and findings presented here establish a solid foundation that can be utilized to push the boundaries of what can currently be achieved in particle physics research.

Appendix A

Equations Appendix

A.1 Likelihood Ratio

Let the parameters θ_0 and θ_1 define two distributions, $p(x \mid \theta_0)$ and $p(x \mid \theta_1)$, as described in Section II.A. of [22]. The goal is to determine or approximate the likelihood ratio

$$\mathcal{L}(x) = \frac{p(x \mid \theta_0)}{p(x \mid \theta_1)} \quad (\text{A.1})$$

between the two distributions.

Consider the general loss functional that depends on a learnable function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and rescaling functions $A : \mathbb{R} \rightarrow \mathbb{R}$ and $B : \mathbb{R} \rightarrow \mathbb{R}$:

$$L[f] = - \int dx (p(x \mid \theta_0)A(f(x)) + p(x \mid \theta_1)B(f(x))). \quad (\text{A.2})$$

We can take the functional derivative of the loss functional to show that the extremum can be transformed to obtain the likelihood ratio:

$$\frac{\delta L}{\delta f} = - \frac{\partial}{\partial f} (p(x \mid \theta_0)A(f(x)) + p(x \mid \theta_1)B(f(x))) \quad (\text{A.3})$$

$$= -(p(x \mid \theta_0)A'(f(x)) \cdot f'(x) + p(x \mid \theta_0)B'(f(x)) \cdot f'(x)) \quad (\text{A.4})$$

$$= 0 \Leftrightarrow -\frac{B'(f(x))}{A'(f(x))} = \frac{p(x \mid \theta_0)}{p(x \mid \theta_1)} = \mathcal{L}(x). \quad (\text{A.5})$$

Appendix B

Code Appendix

B.1 Classifier Model Code

```
1 def DeepSetsAttClass(num_feat, num_heads=4, num_transformer=4, projection_dim
   =32):
2     inputs = Input((None, num_feat))
3     masked_inputs = layers.Masking(mask_value=0.0, name='Mask')(inputs)
4
5     masked_features = TimeDistributed(Dense(projection_dim, activation=None))(
masked_inputs)
6     tdd = TimeDistributed(Dense(projection_dim, activation=None))(
masked_features)
7     tdd = TimeDistributed(layers.LeakyReLU(alpha=0.01))(tdd)
8     encoded_patches = TimeDistributed(Dense(projection_dim))(tdd)
9
10    for _ in range(num_transformer):
11        x1 = layers.LayerNormalization(epsilon=1e-6)(encoded_patches)
12        attention_output = layers.MultiHeadAttention(num_heads=num_heads,
key_dim=projection_dim//num_heads, dropout=0.1)(x1, x1)
13        x2 = layers.Add()([attention_output, encoded_patches])
```

```

14
15     x3 = layers.LayerNormalization(epsilon=1e-6)(x2)
16     x3 = layers.Dense(4*projection_dim, activation="gelu")(x3)
17     x3 = layers.Dense(projection_dim, activation="gelu")(x3)
18     encoded_patches = layers.Add()([x3, x2])
19
20     representation = layers.LayerNormalization(epsilon=1e-6)(encoded_patches)
21     pooled = layers.GlobalAvgPool1D()(representation)
22     representation = Dense(2*projection_dim, activation=None)(pooled)
23     representation = layers.Dropout(0.1)(representation)
24     representation = layers.LeakyReLU(alpha=0.01)(representation)
25
26     outputs = Dense(1, activation='sigmoid')(representation)
27
28     model = Model(inputs=inputs, outputs=outputs)
29     return model

```

Listing B.1: Python code for Deep Sets with Attention originally created by Dr. Vinicius Mikuni.

B.2 Diffusion Generative Model Code

```

1 import numpy as np
2 import tensorflow as tf
3 from tensorflow import keras
4 from tensorflow.keras import layers, Input
5 import time
6 import utils
7 from tensorflow.keras.losses import mse, mae
8 import tensorflow_probability as tfp

```

```

9 from architecture import DeepSetsAtt, Resnet
10 import gc
11 from tensorflow_probability.python.math.diag_jacobian import diag_jacobian
12
13 class GSGM(keras.Model):
14     """Score based generative model"""
15     def __init__(self, num_feat, num_jet, num_embed=64, projection_dim=64,
16 num_layer=4, num_head=2, num_part=50, name='GSGM'):
17         super(GSGM, self).__init__()
18         self.num_feat = num_feat
19         self.num_jet = num_jet
20         self.num_embed = num_embed
21         self.max_part = num_part
22         self.ema = 0.999
23         self.shape = (-1, 1, 1)
24         self.num_steps = 256
25         self.loss_tracker = keras.metrics.Mean(name="loss")
26         inputs_time = Input((1), name='input_time')
27         inputs_jet = Input((self.num_jet), name='input_jets')
28         inputs_particles = Input((None, self.num_feat), name='input_particles')
29
30         inputs_mask = Input((None, 1), name='input_mask')
31         outputs = DeepSetsAtt(self.FF(inputs_particles), inputs_time,
32 inputs_jet, inputs_mask, num_feat=self.num_feat, num_heads=num_head,
33 num_transformer=num_layer, projection_dim=projection_dim)
34
35         self.model_part = keras.Model(inputs=[inputs_particles, inputs_jet,
36 inputs_time, inputs_mask], outputs=outputs)
37
38         outputs = Resnet(inputs_jet, inputs_time, self.num_jet, num_layer=3,
39 projection_dim=2*projection_dim)
40
41         self.model_jet = keras.Model(inputs=[inputs_jet, inputs_time], outputs
42 =outputs)

```



```

33     self.ema_jet = keras.models.clone_model(self.model_jet)
34     self.ema_part = keras.models.clone_model(self.model_part)
35
36     @property
37     def metrics(self):
38         return [self.loss_tracker]
39
40     def prior_sde(self, dimensions):
41         return tf.random.normal(dimensions, dtype=tf.float32)
42
43     def FF(self, features, expand=False):
44         max_proj = 8
45         min_proj = 6
46         freq = tf.range(start=min_proj, limit=max_proj, dtype=tf.float32)
47         freq = 2. ** (freq) * 2 * np.pi
48         x = features
49         freq = tf.tile(freq[None, :], (1, tf.shape(x)[-1]))
50         h = tf.repeat(x, max_proj - min_proj, axis=-1)
51         if expand:
52             angle = h * freq[None, :]
53         else:
54             angle = h * freq
55         h = tf.concat([tf.math.sin(angle), tf.math.cos(angle)], -1)
56         return tf.concat([features, h], -1)
57
58     @tf.function
59     def get_logsnr_alpha_sigma(self, time, shape=None, eps=1e-9):
60         alpha = tf.sqrt(1.0 - tf.clip_by_value(time, eps, 1.0))
61         sigma = tf.sqrt(tf.clip_by_value(time, eps, 1.0))
62         logsnr = 2 * tf.math.log(alpha / sigma)
63         if shape is not None:

```

```

64         logsnr = tf.reshape(tf.cast(logsnr, tf.float32), shape)
65         alpha = tf.reshape(tf.cast(alpha, tf.float32), shape)
66         sigma = tf.reshape(tf.cast(sigma, tf.float32), shape)
67         return logsnr, alpha, sigma
68
69     @tf.function
70     def eval_model(self, model, x, t, jet=None, mask=None):
71         if jet is None:
72             score = model([x, t])
73         else:
74             score = model([x * mask, jet, t, mask]) * mask
75         return score
76
77     @tf.function
78     def get_score(self, model, x, t, jet=None, mask=None, const_shape=None):
79         v = self.eval_model(model, x, t, jet, mask)
80         _, alpha, sigma = self.get_logsnr_alpha_sigma(t, shape=const_shape)
81         score = x - v * alpha / sigma
82         return score
83
84     @tf.function
85     def train_step(self, inputs):
86         batch_size = tf.shape(inputs['input_jets'])[0]
87         random_t = tf.random.uniform((batch_size, 1))
88         logsnr, alpha, sigma = self.get_logsnr_alpha_sigma(random_t)
89         with tf.GradientTape() as tape:
90             eps = tf.random.normal((tf.shape(inputs['input_particles'])),
dtype=tf.float32) * inputs['input_mask'][:, :, None]
91             perturbed_x = alpha[:, None] * inputs['input_particles'] + eps *
sigma[:, None]
92             v_pred_part = self.model_part([perturbed_x, inputs['input_jets'],

```

```

random_t, inputs['input_mask']))
93         v_pred_part = tf.reshape(v_pred_part, (tf.shape(v_pred_part)[0],
-1))
94         v = alpha[:, None] * eps - sigma[:, None] * inputs['
input_particles']
95         v = tf.reshape(v, (tf.shape(v)[0], -1))
96         loss_part = tf.reduce_sum(tf.square(v - v_pred_part)) / (3 * tf.
reduce_sum(inputs['input_mask']))
97         eps = tf.random.normal((tf.shape(inputs['input_jets'])), dtype=tf.
float32)
98         perturbed_x = alpha * inputs['input_jets'] + eps * sigma
99         v_pred_jet = self.model_jet([perturbed_x, random_t])
100        v = alpha * eps - sigma * inputs['input_jets']
101        loss_jet = mse(v_pred_jet, v)
102        loss = loss_part + loss_jet
103        trainable_variables = self.model_jet.trainable_variables + self.
model_part.trainable_variables
104        self.optimizer.minimize(loss, trainable_variables, tape=tape)
105        self.loss_tracker.update_state(loss)
106        for weight, ema_weight in zip(self.model_jet.weights, self.ema_jet.
weights):
107            ema_weight.assign(self.ema * ema_weight + (1 - self.ema) * weight)
108        for weight, ema_weight in zip(self.model_part.weights, self.ema_part.
weights):
109            ema_weight.assign(self.ema * ema_weight + (1 - self.ema) * weight)
110        return {
111            "loss": self.loss_tracker.result(),
112            "loss_part": tf.reduce_mean(loss_part),
113            "loss_jet": tf.reduce_mean(loss_jet),
114        }
115

```

```

116     @tf.function
117     def test_step(self, inputs):
118         batch_size = tf.shape(inputs['input_jets'])[0]
119         random_t = tf.random.uniform((batch_size, 1))
120         logsnr, alpha, sigma = self.get_logsnr_alpha_sigma(random_t)
121         eps = tf.random.normal((tf.shape(inputs['input_particles'])), dtype=tf
122             .float32) * inputs['input_mask'][:, :, None]
123         perturbed_x = alpha[:, None] * inputs['input_particles'] + eps * sigma
124            [:, None]
125         v_pred_part = self.model_part([perturbed_x, inputs['input_jets'],
126             random_t, inputs['input_mask']])
127         v_pred_part = tf.reshape(v_pred_part, (tf.shape(v_pred_part)[0], -1))
128         v = alpha[:, None] * eps - sigma[:, None] * inputs['input_particles']
129         v = tf.reshape(v, (tf.shape(v)[0], -1))
130         loss_part = tf.reduce_sum(tf.square(v - v_pred_part)) / (3 * tf.
131             reduce_sum(inputs['input_mask']))
132         eps = tf.random.normal((tf.shape(inputs['input_jets'])), dtype=tf.
133             float32)
134         perturbed_x = alpha * inputs['input_jets'] + eps * sigma
135         v_pred_jet = self.model_jet([perturbed_x, random_t])
136         v = alpha * eps - sigma * inputs['input_jets']
137         loss_jet = mse(v_pred_jet, v)
138         loss = loss_part + loss_jet
139         self.loss_tracker.update_state(loss)
140         return {
141             "loss": self.loss_tracker.result(),
142             "loss_part": tf.reduce_mean(loss_part),
143             "loss_jet": tf.reduce_mean(loss_jet),
144         }
145     @tf.function

```

```

142     def call(self, x):
143         return self.model(x)
144
145     def get_likelihood(self, part, jet, mask):
146         start = time.time()
147         ll_jet, normal_jet = self.Likelihood(jet, self.ema_jet, data_shape=[
jet.shape[0], self.num_jet], const_shape=[-1, 1])
148         end = time.time()
149         print("Time for calculating the likelihood of {} events is {} seconds"
.format(jet.shape[0], end - start))
150         start = time.time()
151         ll_part, normal_data = self.Likelihood(part, self.ema_part, data_shape
=[part.shape[0], self.max_part, self.num_feat], const_shape=self.shape,
jet=jet, mask=mask)
152         end = time.time()
153         print("Time for calculating the likelihood of {} events is {} seconds"
.format(jet.shape[0], end - start))
154         return ll_part, ll_jet, normal_data * mask, normal_jet
155
156     def generate(self, nevts):
157         start = time.time()
158         jet_info, normal_jet = self.ODESampler(nevts, self.ema_jet, data_shape
=[nevts, self.num_jet], const_shape=[-1, 1])
159         end = time.time()
160         print("Time for sampling {} events is {} seconds".format(nevts, end -
start))
161         nparts = np.expand_dims(np.clip(utils.revert_npart(jet_info[:, -1]),
2, self.max_part), -1)
162         mask = np.expand_dims(np.tile(np.arange(self.max_part), (nparts.shape
[0], 1)) < np.tile(nparts, (1, self.max_part))), -1)
163         start = time.time()

```

```

164     parts, normal_data = self.ODESampler(nevts, self.ema_part, data_shape
=[nevts, self.max_part, self.num_feat], const_shape=self.shape, jet=
jet_info, mask=tf.convert_to_tensor(mask, dtype=tf.float32))
165     end = time.time()
166     print("Time for sampling {} events is {} seconds".format(nevts, end -
start))
167     return parts.numpy() * mask, jet_info.numpy(), normal_data * mask,
normal_jet
168
169     def get_sde(self, time, shape=None):
170         logsnr, alpha, sigma = self.get_logsnr_alpha_sigma(time)
171         f = 1.0 / (2 * alpha ** 2)
172         g2 = 1.0 / (alpha ** 2)
173         if shape is None:
174             shape = self.shape
175         f = tf.reshape(f, shape)
176         g2 = tf.reshape(g2, shape)
177         return tf.cast(f, tf.float32), tf.cast(g2, tf.float32)
178
179     def ODESampler(self, nevts, model, data_shape=None, const_shape=None, jet=
None, mask=None, atol=1e-5, eps=1e-5):
180         from scipy import integrate
181         init_x = self.prior_sde(data_shape)
182         shape = init_x.shape
183         @tf.function
184         def score_eval_wrapper(sample, time_steps, jet=None, mask=None):
185             sample = tf.cast(tf.reshape(sample, shape), tf.float32)
186             time_steps = tf.reshape(time_steps, (sample.shape[0], 1))
187             score = self.get_score(model, sample, time_steps, jet, mask,
const_shape)
188             return tf.reshape(score, [-1])

```

```

189     def ode_func(t, x):
190         time_steps = np.ones((shape[0],)) * t
191         f, g2 = self.get_sde(t, shape=(-1))
192         return f * x - 0.5 * g2 * score_eval_wrapper(x, time_steps, jet,
mask).numpy()
193     res = integrate.solve_ivp(ode_func, (1.0 - eps, eps), tf.reshape(
init_x, [-1]).numpy(), rtol=atol, atol=atol, method='RK23')
194     print(f"Number of function evaluations: {res.nfev}")
195     sample = tf.reshape(res.y[:, -1], shape)
196     return sample, init_x
197
198     def Likelihood(self, sample, model, data_shape=None, const_shape=None, jet
=None, mask=None, atol=1e-5, rtol=5e-4, eps=1e-5, exact=True):
199         from scipy import integrate
200         gc.collect()
201         batch_size = sample.shape[0]
202         shape = sample.shape
203         if mask is None:
204             N = np.prod(shape[1:])
205         else:
206             N = np.sum(self.num_feat * mask, (1, 2))
207         def prior_likelihood(z):
208             shape = z.shape
209             return -N / 2. * np.log(2 * np.pi) - np.sum(z.reshape((shape[0],
-1)) ** 2, -1) / 2.
210         @tf.function
211         def divergence_eval_wrapper(sample, time_steps, jet=None, mask=None):
212             sample = tf.cast(tf.reshape(sample, shape), tf.float32)
213             time_steps = tf.reshape(time_steps, (sample.shape[0], 1))
214             f, g2 = self.get_sde(time_steps, shape=const_shape)
215             epsilons = tfp.random.rademacher(sample.shape, dtype=tf.float32)

```

```

216         if mask is not None:
217             sample *= mask
218             epsilons *= mask
219         if exact:
220             fn = lambda x: f * x - 0.5 * g2 * self.get_score(model, x,
time_steps, jet, mask, const_shape)
221             pred, diag_jac = diag_jacobian(xs=sample, fn=fn, sample_shape
=[batch_size])
222             if isinstance(pred, list):
223                 pred = pred[0]
224                 if isinstance(diag_jac, list):
225                     diag_jac = diag_jac[0]
226             return tf.reshape(pred, [-1]), -tf.reduce_sum(tf.reshape(
diag_jac, (batch_size, -1)), -1)
227         else:
228             with tf.GradientTape(persistent=False,
watch_accessed_variables=False) as tape:
229                 tape.watch(sample)
230                 score = self.get_score(model, sample, time_steps, jet,
mask, const_shape)
231                 drift = f * sample - 0.5 * g2 * score
232                 jvp = tf.cast(tape.gradient(drift, sample, epsilons), tf.
float32)
233             return tf.reshape(drift, [-1]), -tf.reduce_sum(tf.reshape(jvp
* epsilons, (batch_size, -1)), -1)
234         def ode_func(t, x):
235             time_steps = np.ones((batch_size,)) * t
236             sample = x[:-batch_size]
237             logp = x[-batch_size:]
238             sample_grad, logp_grad = divergence_eval_wrapper(sample,
time_steps, jet, mask)

```



```

239         return np.concatenate([sample_grad, logp_grad], axis=0)
240         init_x = np.concatenate([sample.reshape([-1]), np.zeros((batch_size,))
241 ], 0)
242         res = integrate.solve_ivp(ode_func, (eps, 1.0 - eps), init_x, t_eval
243 =[1.0 - eps], rtol=rtol, atol=atol, method='RK45')
244         print(f"Number of function evaluations: {res.nfev}")
245         zp = res.y[:, -1]
246         z = zp[:-batch_size].reshape(shape)
247         if mask is not None:
248             z *= mask
249             delta_logp = zp[-batch_size:].reshape(batch_size)
250             prior_logp = prior_likelihood(z)
251             return (prior_logp - delta_logp), z
252
253     @tf.function
254     def DDPMSSampler(self, nevt, model, data_shape=None, const_shape=None, jet
255 =None, mask=None):
256         batch_size = nevt
257         x = self.prior_sde(data_shape)
258         for time_step in tf.range(self.num_steps, 0, delta=-1):
259             t = tf.ones((batch_size, 1), dtype=tf.int32) * time_step / self.
260 num_steps
261             logsnr, alpha, sigma = self.get_logsnr_alpha_sigma(t, shape=
262 const_shape)
263             logsnr_, alpha_, sigma_ = self.get_logsnr_alpha_sigma(tf.ones((
264 batch_size, 1), dtype=tf.int32) * (time_step - 1) / self.num_steps, shape=
265 const_shape)
266             v = self.eval_model(model, x, t, jet, mask)
267             mean = alpha * x - sigma * v
268             eps = v * alpha + x * sigma
269             x = alpha_ * mean + sigma_ * eps

```

```
263     return mean
```

Listing B.2: Python code for the Diffusion Generative Model and Likelihood Generation created by Dr. Vinicius Mikuni.

Bibliography

- [1] Wikipedia, *File:standard model of elementary particles — wikipedia, the free encyclopedia* (2024), [Online; accessed 27-April-2024], URL https://en.wikipedia.org/wiki/File:Standard_Model_of_Elementary_Particles.svg.
- [2] E. Rudenko, *Deep Learning vs Machine Learning or How AI Benefits Business – NIX United — nix-united.com*, <https://nix-united.com/blog/artificial-intelligence-vs-machine-learning-vs-deep-learning> [Accessed 28-04-2024].
- [3] S. Amidi, *Cs 229 - supervised learning cheatsheet*, <https://stanford.edu/~shervine/teaching/cs-229/cheatsheet-supervised-learning>, accessed: 2024-04-28.
- [4] D. Shah, *Cross Entropy Loss: Intro, Applications, Code — v7labs.com*, <https://www.v7labs.com/blog/cross-entropy-loss-guide> (2023), [Accessed 28-04-2024].
- [5] A. N. Kumar, *Mean Normalization and Feature Scaling—A simple explanation — medium.com*, <https://medium.com/analytics-vidhya/mean-normalization-and-feature-scaling> (2021), [Accessed 28-04-2024].
- [6] J. Peng, E. Jury, P. Dönnies, and C. Ciurtin, *Frontiers in Pharmacology* **12** (2021).

- [7] A. Ostwal, *High Variance to High Bias via “Perfection”* — *towardsdatascience.com*,
<https://towardsdatascience.com/high-variance-to-high-bias-via-perfection-fd69de3a>
 [Accessed 28-04-2024].
- [8] H. Cao, C. Tan, Z. Gao, Y. Xu, G. Chen, P.-A. Heng, and S. Z. Li, ArXiv **abs/2209.02646** (2022), URL
<https://api.semanticscholar.org/CorpusID:252090040>.
- [9] *An Introduction to Diffusion Models for Machine Learning* — *encord.com*,
<https://encord.com/blog/diffusion-models/>, [Accessed 28-04-2024].
- [10] M. M. Waldrop, *News Feature: What are the limits of deep learning?*
 — *ncbi.nlm.nih.gov*, <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC6347705/>
 (2019), [Accessed 29-04-2024].
- [11] G. Kasieczka, T. Plehn, A. Butter, K. Cranmer, D. Debnath, B. M. Dillon, M. Fairbairn,
 D. A. Faroughy, W. Fedorko, C. Gay, et al. (2019), **arXiv:1902.09914**.
- [12] D. H. Perkins, *Introduction to High Energy Physics* (Cambridge University Press, Cam-
 bridge, 2000), 4th ed., ISBN 9780521621960.
- [13] ATLAS Collaboration, *Introduction to atlas* (2020), accessed: 2023-05-07, URL
https://atlasopendata.docs.cern.ch/docs/documentation/introduction/introduction_A7
- [14] National Center for Biotechnology Information, *Ncbi bookshelf* (2020), accessed: 2023-
 05-07, URL <https://www.ncbi.nlm.nih.gov/books/NBK232475/>.
- [15] Wikipedia, *File:standard model – all feynman diagram vertices* — *wikipedia, the free
 encyclopedia* (2024), [Online; accessed 27-April-2024], URL
- [16] M. Jazouli, K. Lekouch, A. Wifaya, and L. Bouirden (2011).

- [17] Radasoft, *How particle accelerators work: From linac to synchrotron* (2024), accessed: 2024-05-08, URL <https://www.radasoft.net/blog/how-particle-accelerators-work-from-linac-to-synch>
- [18] Wikipedia Contributors, *Large hadron collider* (2024), accessed: 2024-05-08, URL https://en.wikipedia.org/wiki/Large_Hadron_Collider.
- [19] Analytics Vidhya, *Common loss functions in machine learning for classification model* (2024), accessed: 2024-05-08, URL <https://medium.com/analytics-vidhya/common-loss-functions-in-machine-learning-for>
- [20] Great Learning, *Types of neural networks* (2024), accessed: 2024-05-08, URL <https://www.mygreatlearning.com/blog/types-of-neural-networks/>.
- [21] GeeksforGeeks, *Machine learning frameworks* (2024), accessed: 2024-05-08, URL <https://www.geeksforgeeks.org/machine-learning-frameworks/>.
- [22] S. Rizvi, M. Pettee, and B. Nachman, *Learning likelihood ratios with neural network classifiers* (2023), arXiv:2305.10500.
- [23] G. Kasieczka, T. Plehn, J. Thompson, and M. Russel, *Top Quark Tagging Reference Dataset* — zenodo.org, <https://zenodo.org/records/2603256> (2019), [Accessed 29-04-2024].